

Sztuczna Inteligencja

w pracy Testera

– Koncepcje, Narzędzia i Metodologie

Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy

Mateusz Kulesza

- Solution Architect
- Senior Software Developer
- Consultant and Trainer

Uczestnik

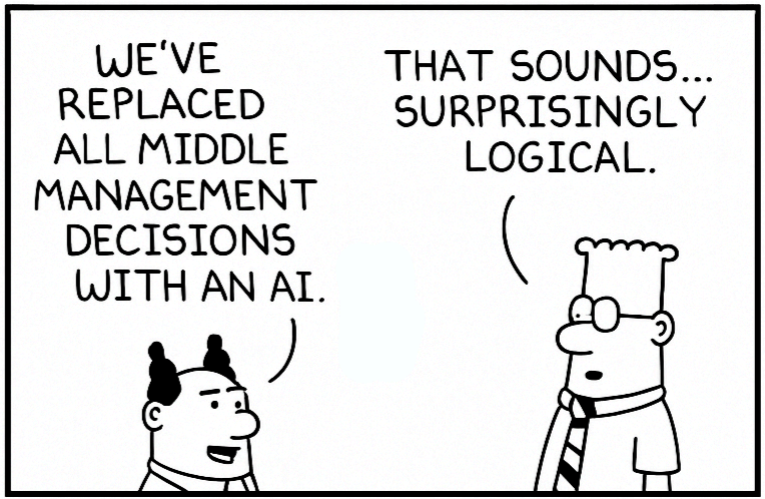
czyli "Ty"

Kilka szybkich pytań:

- Twoje codzienne obowiązki?
- Technologie i background?
- Poprzednie doświadczenie?
- Twoje cele na to szkolenie?
- Określone obszary lub tematy?

Rzeczywistość

w czasach AI



Comic generated by ChatGPT in the style of Scott Adams.

Jak odróżniać marketing AI od twardej inżynierii?

🚀 Hype

– vs. Produktywna Rzeczywistość

🤔 **Dlaczego w ogóle jest hype?**

- **LLM-y wyglądają jak „inteligencja”** → płynny język ≠ głęboka wiedza
- **Marketing producentów modeli**: „agent”, „rozumowanie”, „samodzielność”
- **Media i społeczność** wzmacniają narrację „AI zastąpi programistów”
- **Większość testów jest kontentowych, nie inżynieryjnych**

  – screenshoty promptów zamiast rzeczywistych pipelines

🎯 Produktywna rzeczywistość

– co faktycznie działa dziś?

AI = komponent w procesie, nie mózg systemu

- Model jest **funkcją probabilistyczną**, a nie deterministyczną
- Wymaga otoczenia "prawdziwego" systemu:
  - walidacja I/O
  - guardrails + kompensacje
  - kontekst + filtracja danych
  - obserwowalność, trace
  - governance
  - i wiele innych ...

Co działa 100% przewidywalnie?

– większość AI nie jest przewidywalne!

Aby mieć gwarancje wyników:

- **Kod generowany pod nadzorem** → oszczędza 30–70% czasu
- **AI jako parser / konwerter / etykieciarz** danych
- **Embeddingi + wektorowe wyszukiwanie**
- **Pipeline'y złożone z wielu kroków**, np.
  - OCR → extraction → LLM → decision engine
- **Generowanie dokumentów technicznych, diagramów, testów** - z szablonów
- **Asystenci w IDE (Cursor, Copilot++)**

AI jako probabilistyczna część deterministycznego systemu

Co działa, ale z ograniczeniami?

– AI Bezpośredio, ale pod kontrolą

- **Autonomous Agents** → w praktyce 1–3 zadania, nie łańcuchy 100 kroków
- **Fine-tuning** → sens only w przypadku dużych wolumenów i stabilnej domeny
- **AI in the loop** → AI jako element procesu
- **Human in the loop** → AI jako "asystent", człowiek weryfikuje
- **Retrieval Augmented Generation**, ale tylko z:
  - chunkingiem dopasowanym do schematu danych
  - filtrami domenowymi
  - guardrails
  - testami regresyjnymi promptów
  - itd...

Co **NIE DZIAŁA** tak, jak mówi hype

Marketingowa "ściema" i **science-fiction**:

- „AI zrozumie kontekst firmy i będzie jak pracownik”
- „Agent sam ogarnie wieloetapowe zadanie bez kontroli”
- „LLM zastąpi architekta”
- „Model wygeneruje od razu poprawny kod produkcyjny”
- „Dzięki Vibe-Coding nie trzeba rozumieć kodu”
- „RAG rozwiąże wszystkie problemy z wiedzą firmy”

AI nie "myśli", tylko **przewiduje** co będzie dalej:

- Model jest trenowany na opiniach co jest "dobrze" a co "źle"
- Im lepsze dostaje dane wejściowe tym lepsze predykcje
- Im lepiej kontrolowany proces tym mniej niespodzianek

🛠️ Rzeczywiste wyzwania,

o których hype nie mówi

- **Brak transakcyjności** → model może „magicznie” zmienić odpowiedź
- **Halucynacje w 2–10% przypadków** nawet w top modelach
- **Drift danych biznesowych** → model staje się nieaktualny po 3–9 miesiącach
- **Zarządzanie danymi osobowymi** (RODO, AI Act)
- **Compliance, governance, audytowalność**
- **Black box problem** → dlaczego model podjął taką decyzję?
- **Idempotencja?** → tylko pozorna
- **Brak gwarancji deterministycznej interpretacji polecenia**

AI w pracy testera

podstawy bez hype’u

Cel: rozumieć, gdzie AI pomaga, gdzie kłamie, i jak pisać prompty, które dają użyteczny wynik.

AI

nie jest:

- magicznym testerem
- źródłem prawdy
- zastępstwem myślenia domenowego
- gwarancją poprawnej odpowiedzi
- narzędziem „wrzuć wymagania → dostaniesz testy idealne”

jest:

- generatorem propozycji
- asystentem do przyspieszania pracy
- narzędziem do eksploracji, streszczania, transformacji
- partnerem do pierwszej wersji, nie do ostatecznej decyzji

Hype vs praktyka

Hype

> „AI napisze wszystkie test case'y za mnie."

Praktyka

AI może pomóc wygenerować **pierwszą wersję**, ale tester musi sprawdzić:

- czy pokrywa wymagania
- czy zna ograniczenia systemu
- czy nie wymyśliła funkcji
- czy uwzględnia ryzyka biznesowe
- czy przypadki są wykonalne

Typowe błędy myślenia o AI

„Brzmi dobrze” ≠ „jest prawdziwe”

Model może pisać pewnie nawet wtedy, gdy się myli.

„Długi prompt” ≠ „dobry prompt”

Dobry prompt jest konkretny, uporządkowany i ma jasny cel.

„AI zna projekt” ≠ „AI zna projekt”

AI zna tylko to, co dostała w kontekście albo co ma w parametrach modelu.

Model a Czat

– z kim tak na prawdę rozmawiamy?

Model

Silnik przewidujący tekst.

Przykład: GPT-5.5, Claude, Gemini, Llama.

Chat

Interfejs rozmowy z modelem.

Przykład: wpisujesz pytanie, dostajesz odpowiedź.

Asystant

Model + instrukcje + narzędzia + pamięć/kontekst.

Może mieć:

- rolę: „jesteś QA Leadem”
- zasady: „odpowiadaj po polsku, zwięźle”
- pliki projektowe
- narzędzia: Gmail, Jira, Drive, przeglądarka
- format odpowiedzi

Przykład

> „Jesteś asystentem QA. Analizujesz user stories i tworzysz checklisty testowe.”

Agent

Asystant, który może wykonywać kroki w celu osiągnięcia zadania.

Różnica

- Chat odpowiada.
- Agent planuje, używa narzędzi, sprawdza wynik, poprawia.

Przykład

> „Przejdź przez formularz, wypełnij dane z tabeli, kliknij Save, sprawdź czy rekord pojawił się na liście.”

Model / chat / asystant / agent

— porównanie

Poziom	Co robi	Przykład QA
Model	generuje tekst	„Napisz przypadki testowe”
Chat	rozmowa z modelem	doprecyzowanie wymagań
Asystant	model + instrukcje	QA helper z szablonem test case
Agent	wykonuje kroki	automatycznie testuje formularz

Token

Token to fragment tekstu używany przez model.

Tokenem może być:

- słowo
- część słowa
- znak interpunkcyjny
- fragment kodu

Przykład

```
Użytkownik loguje się do aplikacji
```

Model nie „czyta zdań” jak człowiek. Operuje na sekwencji tokenów.

https://platform.openai.com/tokenizer

Next-token prediction

Model przewiduje kolejny token na podstawie poprzednich tokenów.

Uproszczenie

> „Dla podanego kontekstu, jaki fragment tekstu jest najbardziej prawdopodobny dalej?”

Przykład

Prompt:

> „Przypadki testowe dla logowania: 1.”

Model może kontynuować:

> „Poprawne logowanie użytkownika z prawidłowym loginem i hasłem.”

Kontekst

Kontekst to wszystko, co model widzi podczas generowania odpowiedzi.

W kontekście mogą być:

- Twoja wiadomość
- historia rozmowy
- instrukcje systemowe
- pliki
- wyniki z narzędzi
- fragmenty dokumentacji

Zasada

Model nie korzysta z „całego świata”. Korzysta z tego, co ma dostępne w danym przebiegu.

Okno kontekstu

Okno kontekstu to limit informacji, które model może naraz uwzględnić.

Gdy kontekstu jest za dużo:

- starsze fragmenty mogą wypaść
- model może pomijać szczegóły
- odpowiedzi stają się bardziej ogólne
- rośnie ryzyko pomyłek

Dla testera

Nie wrzucaj „wszystkiego”. Daj modelowi **właściwy wycinek**.

Sesja

Sesja to przebieg rozmowy lub pracy z modelem.

W sesji model może widzieć:

- wcześniejsze wiadomości
- ustalone formaty
- przesłane pliki
- decyzje z rozmowy

Ale:

Sesja nie jest tym samym co trwała pamięć ani pełna znajomość projektu.

Bezstanowość

Model bazowo jest bezstanowy.

To znaczy:

Każda odpowiedź powstaje z aktualnie podanego kontekstu.

Przykład

Jeśli w nowej rozmowie napiszesz:

> „Wygeneruj testy do tego modułu”

Model nie wie, o który moduł chodzi, jeśli nie dostanie kontekstu.

Cutoff date

Model ma granicę wiedzy treningowej.

Konsekwencje

Może nie znać:

- najnowszych wersji narzędzi
- aktualnych bibliotek
- nowych przepisów
- zmian w projekcie
- ostatnich decyzji zespołu

Przykład QA

> „Napisz testy dla najnowszej wersji Playwrighta”

Bez sprawdzenia dokumentacji model może użyć nieaktualnej składni.

Niedeterminizm

Ten sam prompt może dać różne odpowiedzi.

Dlaczego?

Model wybiera kolejne tokeny probabilistycznie.

Skutek

AI nie jest idealnym narzędziem do zadań wymagających 100% powtarzalności bez walidacji.

Dobra praktyka

Dla ważnych zadań: generuj, sprawdzaj, porównuj, waliduj.

Wiedza parametryczna

Wiedza parametryczna to to, czego model „nauczył się” w treningu.

Przykłady

- ogólne wzorce testowania
- typowe klasy równoważności
- techniki BVA
- składnia popularnych języków
- ogólne pojęcia QA

Ryzyko

Może być ogólna, nieaktualna albo niepasująca do projektu.

Wiedza kontekstowa

Wiedza kontekstowa to informacje dostarczone modelowi teraz.

Przykłady

- user story
- acceptance criteria
- screen formularza
- log błędu
- fragment API response
- dokument wymagań
- tabela danych testowych

Zasada

Im bardziej projektowe zadanie, tym ważniejszy kontekst.

Parametryczna vs kontekstowa

Typ wiedzy	Skąd pochodzi	Przykład	Ryzyko
Parametryczna	trening modelu	„jak pisać test case’y”	ogólność
Kontekstowa	prompt/pliki/narzędzia	„nasz formularz ma pola A, B, C”	brak lub zły kontekst

Dla testera

Najlepsze wyniki: ogólna wiedza modelu + konkretny kontekst projektu.

Struktura dobrego promptu

Dobry prompt ma 6 elementów:

1. rola
2. cel
3. kontekst
4. zadanie
5. ograniczenia
6. format odpowiedzi

Prompt

— prosty szablon

```
Rola: Jesteś doświadczonym testerem manualnym.
Cel: Pomóż przygotować testy dla funkcji logowania.
Kontekst: Aplikacja e-commerce. Logowanie emailem i hasłem. Blokada po 5 błędnych próbach.
Zadanie: Wypisz przypadki testowe.
Ograniczenia: Uwzględnij pozytywne, negatywne i brzegowe przypadki.
Format: Tabela: ID, scenariusz, kroki, dane, oczekiwany rezultat, priorytet.
```

Prompt słaby vs dobry

Słaby

> „Napisz testy logowania.”

Dobry

> „Jesteś testerem QA. Na podstawie poniższych acceptance criteria przygotuj testy manualne dla logowania w aplikacji e-commerce. Uwzględnij przypadki pozytywne, negatywne, walidacje, blokadę konta i komunikaty błędów. Zwróć wynik w tabeli.”

Przykład: generowanie test case’ów

Wejście

> Użytkownik może zresetować hasło przez email. Link jest ważny 15 minut. Po użyciu link wygasa.

Zadanie dla AI

- wypisz przypadki testowe
- znajdź edge cases
- zaproponuj dane testowe
- wskaż ryzyka

Przykład: wynik, którego chcemy

Obszar	Przykład testu
pozytywny	reset hasła przez poprawny email
negatywny	email nieistniejącego konta
brzegowy	użycie linku po 15 minutach
bezpieczeństwo	ponowne użycie tego samego linku
UX	jasny komunikat po wysłaniu emaila

Kontekst w plikach

Kontekst można przechowywać jako pliki, które model dostaje przy pracy.

Typowe pliki

- prompt
- instrukcja
- skill
- agent
- szablon odpowiedzi
- dane testowe
- wymagania
- checklisty

Plik: prompt

Prompt to jednorazowe polecenie do wykonania zadania.

Przykład

`prompt-test-cases.md`

```
Na podstawie user story wygeneruj przypadki testowe.
Uwzględnij happy path, błędy, walidacje, edge cases.
Zwróć tabelę: ID, nazwa, kroki, expected result, priorytet.
```

Kiedy używać?

Do powtarzalnych, prostych zadań.

Plik: instrukcja

Instrukcja opisuje stałe zasady pracy.

Przykład

`qa-instructions.md`

```
Zawsze odpowiadaj po polsku.
Nie wymyślaj funkcji, których nie ma w wymaganiach.
Oznacz niepewności jako „Do wyjaśnienia”.
Dla każdego przypadku testowego dodaj priorytet.
```

Kiedy używać?

Gdy chcesz mieć spójny styl i reguły odpowiedzi.

Plik: skill

Skill to instrukcja wykonania konkretnej procedury.

Przykład

`skill-analyze-bug-report.md`

```
1. Streść błąd jednym zdaniem.
2. Wypisz kroki reprodukcji.
3. Wskaż dane wejściowe.
4. Oceń severity i priority.
5. Zaproponuj brakujące informacje.
6. Przygotuj wersję zgłoszenia do Jiry.
```

Kiedy używać?

Dla powtarzalnych workflow QA.

Plik: agent

Agent opisuje rolę, narzędzia i sposób działania.

Przykład

`agent-qa-reviewer.md`

```
Rola: QA Reviewer.
Cel: Sprawdzać user stories przed developmentem.
Narzędzia: pliki wymagań, checklisty, API docs.
Procedura: analizuj wymagania, wypisz luki, wygeneruj pytania, zaproponuj testy.
Format: sekcje: Ryzyka, Pytania, Testy, Dane testowe.
```

Kiedy używać?

Dla większych, wieloetapowych zadań.

Mini-workflow dla testera

1. Wklej wymaganie
2. Dodaj kontekst domenowy
3. Poproś o testy
4. Poproś o edge cases
5. Poproś o ryzyka
6. Sprawdź, co AI wymyśliła
7. Popraw i dopiero wtedy użyj

Ćwiczenie 1

Zadanie

Dostajesz wymaganie:

> Użytkownik może dodać produkt do koszyka. Jeżeli produkt jest niedostępny, przycisk „Dodaj do koszyka” jest nieaktywny.

Napisz prompt, który wygeneruje:

- testy pozytywne
- testy negatywne
- edge cases
- pytania do analityka

Ćwiczenie 2

Popraw prompt

Słaby prompt:

> „Sprawdź wymagania.”

Cel

Przerób go na prompt z elementami:

- rola
- cel
- kontekst
- zadanie
- ograniczenia
- format odpowiedzi

Ćwiczenie 3

Analiza odpowiedzi AI

Model wygenerował test:

> „Sprawdź, czy użytkownik może zapłacić BLIKiem.”

Ale wymagania nie wspominają o BLIK.

Pytania

- co tu poszło źle?
- jaki typ wiedzy model użył?
- jak poprawić prompt?
- jak oznaczyć niepewność?

Najważniejsze zasady

1. AI proponuje, tester decyduje
2. Kontekst jest ważniejszy niż „magiczny prompt”
3. Nie ufaj odpowiedzi bez walidacji
4. Używaj formatów: tabela, lista, checklisty
5. Wymuszaj oznaczanie niepewności

Checklista dobrego użycia AI w QA

- Czy model dostał wymaganie?
- Czy zna ograniczenia systemu?
- Czy dostał format odpowiedzi?
- Czy ma zakaz wymyślania funkcji?
- Czy oznacza niepewności?
- Czy wynik został sprawdzony przez człowieka?
- Czy testy są wykonalne?
- Czy pokrywają ryzyka biznesowe?

Podsumowanie

AI w testowaniu działa najlepiej jako:

- generator pierwszej wersji
- partner do eksploracji
- narzędzie do znajdowania luk
- pomoc w porządkowaniu informacji
- przyspieszacz pracy, nie autopilot jakości

Hasło do zapamiętania

Nie pytaj AI: „zrób za mnie”.
Pytaj: „pomóż mi szybciej myśleć i lepiej sprawdzić”.

Prompt Engineering

jako element integracji systemów z AI

Prompty

jako interfejsy

- Prompt = **API kontraktowe** między systemem a modelem

- **Wejście:**
  - formatowane dane,
  - kontekst,
  - rola,
  - ograniczenia

- **Wyjście:**
  - przewidywalna struktura
  - JSON, listy, schematy

Prompty są częścią **architektury systemu**, nie tylko dodatkiem

Hermetyzacja promptów:

- dedykowane moduły promptów
- spójne konwencje, nazewnictwo, parametry modeli

Prompty jako komponenty:

- „prompt-per-task” → separacja odpowiedzialności
- miks warstw: system → developer → user

Standardy, testowanie, wersjonowanie

Wersjonowanie promptów:

- repozytorium kodu
- PR review → porównanie jakości odpowiedzi
- tagowanie:

  `/prompts/v1/analysis/system-architecture.md`

Testowanie promptów:

- A/B testy jakości odpowiedzi
- regression tests: te same dane

  → stabilność wyjścia
- testy „adversarial”

  → sprawdzanie odporności na injection

Operacjonalizacja:

- SLO: czas odpowiedzi, spójność, topical precision
- monitoring driftu danych i efektu zmian promptów
- walidacja schematu odpowiedzi (JSON Schema)
- limity długości i precyzji odpowiedzi
- obserwacja tokenów → koszty i czas

Szablony

Prompt Templates

query:
„Stwórz ADR dotyczący wyboru CQRS w mikroserwisie płatności.”

context:
„System posiada dużo operacji odczytu, ale ma być zgodny z PSD2 i audytowalny.”

→ model otrzyma spójny kontekst i dostarczy poprawny ADR.

```
SYSTEM:
You are an expert assistant.
Provide validated structured output only.

USER QUERY:
{query}

CONTEXT:
{context}

INSTRUCTIONS:
1. Combine query + context.
2. No hallucinations.
3. If context is irrelevant, ignore it.
4. Missing info → return "missing_data".
```

Po co stosować

template prompt

Standaryzacja pracy

- Każdy prompt ma tę samą strukturę → mniej błędów, spójne rezultaty.
- Łatwiej tworzyć procesy, checklisty i workflow dla zespołu.

Powtarzalność i automatyzacja

- Jeden template może obsłużyć setki przypadków użycia.
- Można go wstrzykiwać do CI/CD, Confluence, Jira, Copilot, pipelines.

Lepsza kontrola jakości

- Wymuszenie formatów (JSON, ADR, RFC, diagramy).
- Minimalizacja halucynacji dzięki kontraktowi wejście/wyjście.

Po co stosować

template prompt

Łatwiejsze testowanie i wersjonowanie

- Prompt jako artefakt → można go testować, porównywać i versionować w Git.
- A/B testy promptów stają się proste.

Większa odporność na błędne dane

- Template pozwala modelowi reagować na braki: zgłosić ryzyka, nie wymyślać.

Hermetyzacja logiki

- Template oddziela *logikę zadania* od *zmiennych danych* (query, context).
- Tak jak w programowaniu: parametr zamiast „wklejania kodu”.

Spójność komunikacji

- W zespołach architektonicznych, analitycznych, dev i business — każdy używa tych samych formatów, produktów wyjściowych i standardów.

Trenowanie

Asystenta AI

Zaufanie do AI

— analogia nowego pracownika

- Model AI jest jak **nowy pracownik**
- Ma:
  - wiedzę ogólną
  - doświadczenie zawodowe
- Nie ma:
  - znajomości naszej firmy
  - kontekstu decyzji
  - wiedzy o ryzykach i priorytetach

Zaufanie **nie jest dane** — jest **budowane procesem**

Dlaczego na początku
nie ufamy odpowiedziom AI?

- Błędy nie wynikają ze „złej woli”
- Najczęstsze przyczyny:
  - brak kontekstu
  - brak zasad
  - brak informacji, czego **nie wolno**

Dokładnie jak u nowego pracownika

jeśli nie ma wytycznych to ...

„jakoś" to zrobi 😟

AI i nowy pracownik — to samo ryzyko

- Obaj:
  - chcą pomóc
  - potrafią analizować
  - mogą popełniać błędy
- Problem:
  - **bez instrukcji będą zgadywać**
- Rozwiązanie:
  - jasno powiedzieć **kiedy dopytać**
  - jasno określić **kiedy powiedzieć „nie wiem”**

Instrukcje = opis roli pracownika

Instrukcje mówią asystentowi:

- Kim jesteś (np. tester, nie decydent)
- Jaki masz cel (analiza, nie decyzja)
- Jak się zachować, gdy:
  - brakuje danych → **zadaj pytania**
  - są niejasności → **zaznacz ryzyko**
  - czegoś nie wiesz → **powiedz to wprost**
- Efekt:
  - mniej zgadywania
  - więcej odpowiedzialnych odpowiedzi

Zgadywać czy dopytać?

- Bez instrukcji:
  - AI „spróbuje pomóc” → **wymyśli**
- Z instrukcją:
  - AI **zatrzymuje się i pyta**
- Dokładnie jak w onboardingu:
  - brak zasad = improwizacja
  - jasne zasady = przewidywalność

Gotowe prompty = procedury pracy

- Prompty działają jak:
  - checklisty
  - procedury (SOP)
  - standardy analizy
- Dają:
  - spójny sposób myślenia
  - powtarzalne wyniki
  - mniejsze ryzyko błędów
- My i „pracownik”:
  - działamy według **tych samych zasad**

Guardrails = granice odpowiedzialności

Guardrails określają:

- ❌ czego **nie wolno**:
  - podejmować decyzji biznesowych
  - zakładać faktów
- ⚠️ co jest ryzykowne:
  - prawo
  - compliance
  - architektura
- ✅ co jest OK:
  - analiza
  - porównanie opcji
  - wskazanie ryzyk
- Jak regulamin pracy w firmie

Jak naprawdę wygląda uczenie się

Skuteczny proces (człowiek i AI):

1. Pokazujesz przykład
2. Wyjaśniasz **dlaczego**
3. Prosisz o wyjaśnienie własnymi słowami
4. Sprawdzasz wynik
5. Tłumaczysz:
   - co jest dobre
   - co złe
   - dlaczego

- Cel: **rozumienie**, nie zgadywanie

Narzędzia = samodzielność

- Dostęp do:
  - dokumentacji
  - systemów
  - danych
- Efekt:
  - pracownik działa samodzielnie
  - AI nie zgaduje, tylko sprawdza
- Bez narzędzi:
  - każdy musi pytać o wszystko

Podsumowanie

Chat lub pracownik:

- muszą poznać:
  - nasz styl pracy
  - nasze cele
- muszą nauczyć się:
  - terminów
  - procesów
  - granic

- na początku:
  - pełna kontrola
- później:
  - wspólne procedury
  - nauka na błędach
- Zaufanie:
  - nie do „inteligencji”
  - do **procesu, który kontrolujemy**

Antywzorce w pracy z AI

(i nowym pracownikiem)

- ❌ „AI powinno samo wiedzieć”
  - brak kontekstu = zgadywanie
- ❌ Brak jasnej roli
  - raz analityk, raz decydent
- ❌ Brak granic
  - AI „pomaga” tam, gdzie nie wolno
- ❌ Kara za powiedzenie „nie wiem”
  - efekt: pewne, ale błędne odpowiedzi
- ❌ Brak informacji zwrotnej
  - te same błędy powtarzane w kółko
- ❌ Traktowanie jednorazowej odpowiedzi jak prawdy
  - brak procesu = brak zaufania

Ćwiczenie — AI Onboarding

Cel:

- Doświadczyć, że **zaufanie do AI buduje się jak do pracownika**

Krok 1 – Nadanie roli:

- Opisz AI:
  - kim jest
  - czego **nie robi**
  - jaki ma cel

Ćwiczenie — AI Onboarding

Krok 2 – Zasady:

- Kiedy ma:
  - dopytać
  - powiedzieć „nie wiem”
  - zaznaczyć ryzyko

Krok 3 – Przykład:

- Pokaż 1 dobry przykład analizy
- Wyjaśnij **dlaczego jest dobry**

Ćwiczenie — AI Onboarding

Krok 4 – Sprawdzenie:

- Poproś AI o podobne zadanie
- Oceń:
  - co zrobiło dobrze
  - co źle
  - dlaczego

Krok 5 – Refleksja:

- Co zmieniło się w jakości odpowiedzi?
- Co trzeba było doprecyzować?
- Jakie guardrails były kluczowe?

Ćwiczenie
— AI Onboarding

Krok 6 - Proces:

Zapisz to jako pliki:

- Instrukcje dla agenta
- Wzór promptów dla kroków
- Checklist - jakość odpowiedzi
- Przykładowe zadanie

Nie pozwól by wiedza "uleciała"

– Ty pamiętasz, nowy czat już nie.

AI to dużo więcej niż ChatGPT

Jest wiele rodzajów sztucznej inteligencji

Predictive Analytics

Przewidywanie na podstawie danych historycznych.

- **Forecasting** – popyt, sprzedaż, ceny
- **Risk scoring** – bankowość, ubezpieczenia
- **Churn prediction** – utrata klientów
- **Anomaly Detection** – fraudy, nieprawidłowości
- **Predictive Maintenance** – awarie maszyn
- **Credit Scoring** – scoring BIK/KNF
- **Pricing Models** – dynamiczne ceny, airline pricing

Predictive Analytics

Przewidywanie na podstawie danych historycznych.

Forecasting

- Prophet, NeuralProphet, ARIMA, GluonTS

Risk / Scoring

- XGBoost, LightGBM, CatBoost

Anomaly Detection

- Isolation Forest, LOF, PyOD, Azure Anomaly Detector

Predictive Maintenance

- Azure ML, AWS SageMaker, TensorFlow, PyTorch

Natural Language Processing

NLP - Rozumienie i generowanie języka naturalnego.

Klasyfikacja tekstu – spam, ryzyka, tonacja

Named Entity Recognition – firmy, osoby, kwoty

Information Extraction – dane z umów, polis, pism sądowych

Intent Detection – routing spraw, call centers

Machine Translation – tłumaczenia techniczne

Sentiment & Emotion Analysis – social listening

Semantic Search – RAG, wektory, chat z dokumentacją

Summarization – skróty dokumentów, protokoły

Natural Language Processing

NLP - Rozumienie i generowanie języka naturalnego.

Klasyfikacja, NER

Modele BERT, spaCy, HuggingFace Transformers

Flair NLP, fastText

Semantic Search / RAG

OpenAI Embeddings, Voyage, Cohere Embed, HuggingFace sentence-transformers

PgVector, Pinecone, Weaviate, ChromaDB, Milvus

Summarization & QA

LLM: GPT-4/5, Claude, Gemini, Llama, Mistral

Pipeline: LangChain, LlamaIndex, OpenAI Assistants API

Machine Translation

MarianMT, M2M-100, NLLB, DeepL API

Computer Vision

AI operująca na obrazach, wideo i sygnałach wizualnych.

- **OCR / ICR** – faktury, KYC, dowody, paszporty
- **Doc Understanding** – klasyfikacja dokumentów, tabelaryzacja
- **Object Detection** – YOLO, DETR, autonomiczne pojazdy
- **Image Segmentation** – medycyna, analizy satelitarne
- **Face Recognition** – dostęp fizyczny
- **Quality Control** – linie produkcyjne
- **Medical Imaging** – RTG, MRI, CT
- **Satellite/Drone Vision** – geodezja, rolnictwo

Computer Vision

AI operująca na obrazach, wideo i sygnałach wizualnych.

- **OCR / ICR**

Tesseract OCR, Google Vision OCR, Azure Form Recognizer, AWS Textract

PaddleOCR, EasyOCR, Kofax

- **Object Detection**

YOLOv5/v7/v8, Detectron2, DETR, OpenCV DNN

Frameworki: PyTorch, TensorFlow, Keras

- **Image Segmentation**

U-Net, Mask R-CNN, Segment Anything (SAM)

- **Face Recognition**

dlib, FaceNet, DeepFace, OpenCV

Speech AI

Audio, głos, mowa, sygnały akustyczne.

- **ASR** – Automatic Speech Recognition
- **Speech-to-Text** – transkrypcje, call center
- **Text-to-Speech** – wirtualni asystenci
- **Speaker Recognition** – biometria głosowa
- **Emotion Detection** – analiza stresu w call center

Speech AI

Audio, głos, mowa, sygnały akustyczne.

Speech-to-Text (ASR)

Whisper, DeepSpeech, Google Speech API, Amazon Transcribe

Text-to-Speech (TTS)

ElevenLabs, Microsoft Neural TTS, Coqui TTS

Speaker Recognition

DeepSpeaker, speechbrain

Generative AI

Tworzenie nowych treści.

- **LLM** – generowanie tekstu, kodu, dokumentacji
- **Code Generation** – Copilot, PR review
- **Image Generation** – obrazy, UI mockupy
- **Video Generation** – animacje, generowanie scen
- **Audio Generation** – muzyka, narracja
- **3D/Modeling** – CAD, VR, gry
- **Diagram Generation** – sekwencje, architektury
- **Agent-based Automation** – agenci wieloetapowi

Generative AI

Tworzenie nowych treści.

Text (LLM)

- OpenAI GPT-4.1/5, Claude 3.x, Gemini 2.x
- Llama 3.x, Mistral 8x7B, Qwen 2

Image Generation

- Stable Diffusion (SDXL, SD 3)
- Midjourney, DALL·E 3/4, Adobe Firefly

Video Generation

- Sora, Pika Labs, RunwayML Gen-2

Audio

- ElevenLabs, Sunō, Tortoise-TTS, Whisper

3D / CAD

- Gaussian Splatting, NerfStudio

Code Generation

- GitHub Copilot, Cursor, Codeium, Aider

Knowledge AI & RAG

Praca na wiedzy, danych, dokumentach, bazach.

- Vector Search
- Retrieval-Augmented Generation
- Enterprise Knowledge Graphs
- Document QA – chatboty domenowe
- Semantic ETL – automatyczne przetwarzanie danych

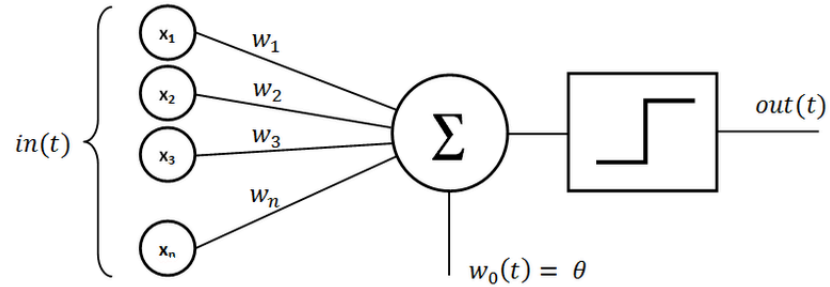
Historia AI

w pigułce

Wczesne sieci neuronowe

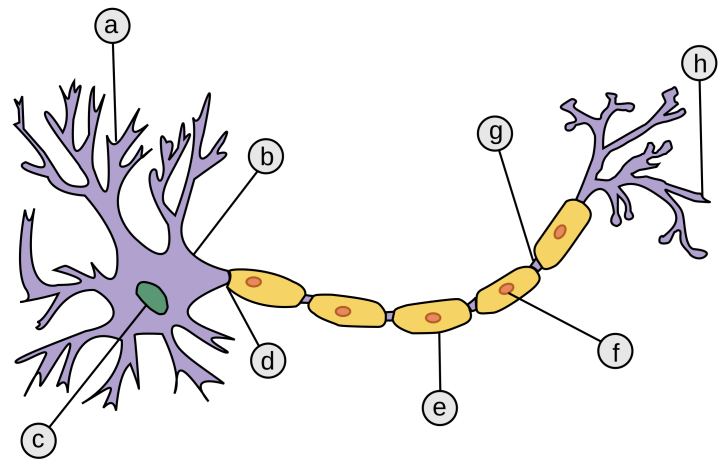
(lata 60–80)

Perceptron - uczenie przez stopniowe dobieranie wag i biasów (progów) tak, by wyniki podawane na wyjściu sieci były coraz bardziej zgodne z wartościami oczekiwanymi



Kamienie milowe:

- perceptron (Rosenblatt, 1957)
- sieci jednokierunkowe → ograniczona moc
- brak efektywnego uczenia wielowarstwowego



Problem:

Minsky, Papert: perceptron nie potrafi XOR - Zima AI

Perceptron – podstawy

Perceptron to najprostszy model sztucznego neuronu używany do liniowej klasyfikacji.

Model matematyczny

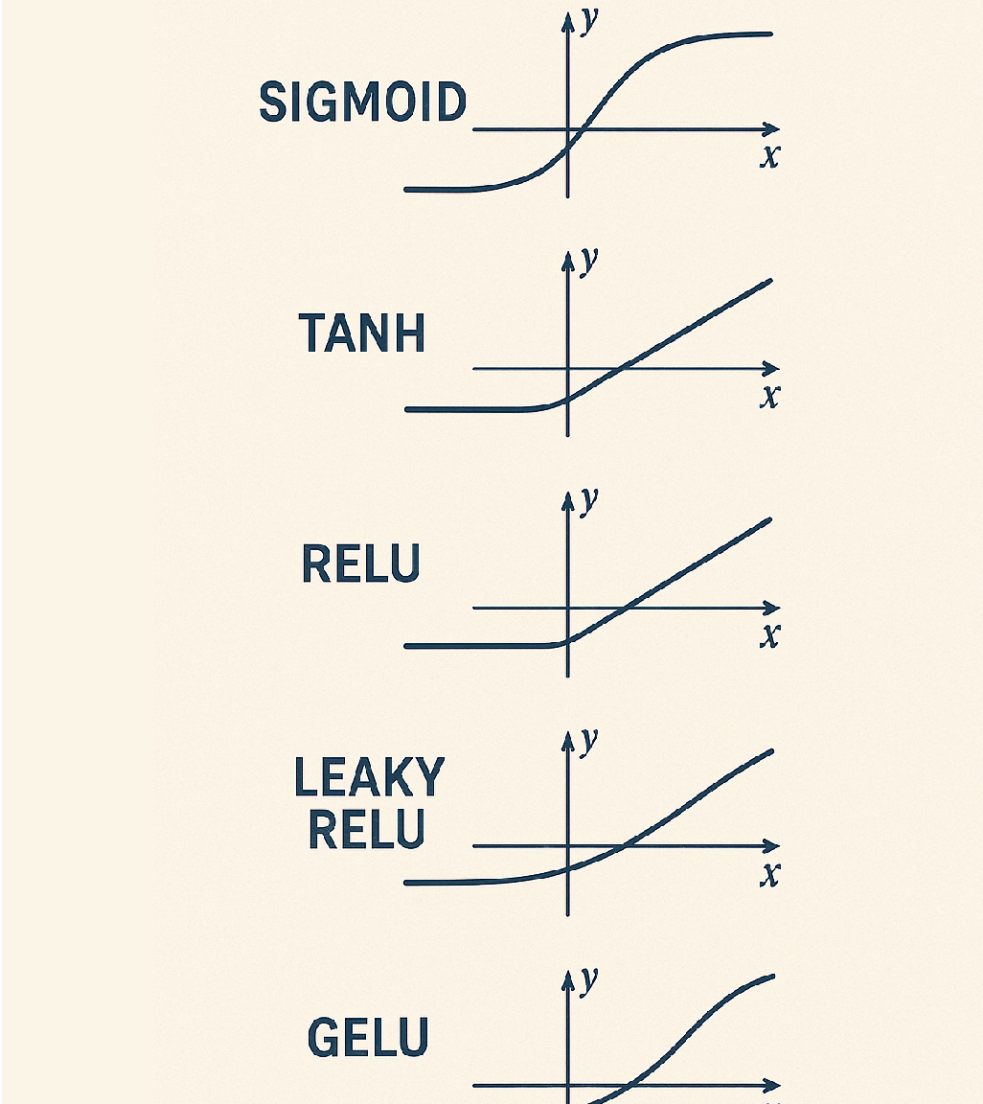
Wejścia: $x_1, x_2, \ldots, x_n$
Wagi: $w_1, w_2, \ldots, w_n$
Bias: b

Obliczenie aktywacji neuronu:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Wyjście po funkcji aktywacji (np. sign):

$$\hat{y} = \text{sign}(z)$$



Backpropagation

(1986)

Przełom:

- Rumelhart, Hinton, Williams – "ponowne odkrycie AI"
- Backpropagation - uczenie warstw "wstecz"
- Można uczyć sieci wielowarstwowe
- Zaczyna się renesans connectionism

Efekt / Problem:

sieci MLP działają, ale nadal brak mocy obliczeniowej

Kroki

1. **Forward pass** – obliczamy wyjście sieci.
2. **Loss** – obliczamy błąd
3. **Propagacja wsteczna** – liczymy pochodne błędu po każdej wadze
4. **Aktualizacja wag**:

$$w_i := w_i - \eta \frac{\partial L}{\partial w_i}$$

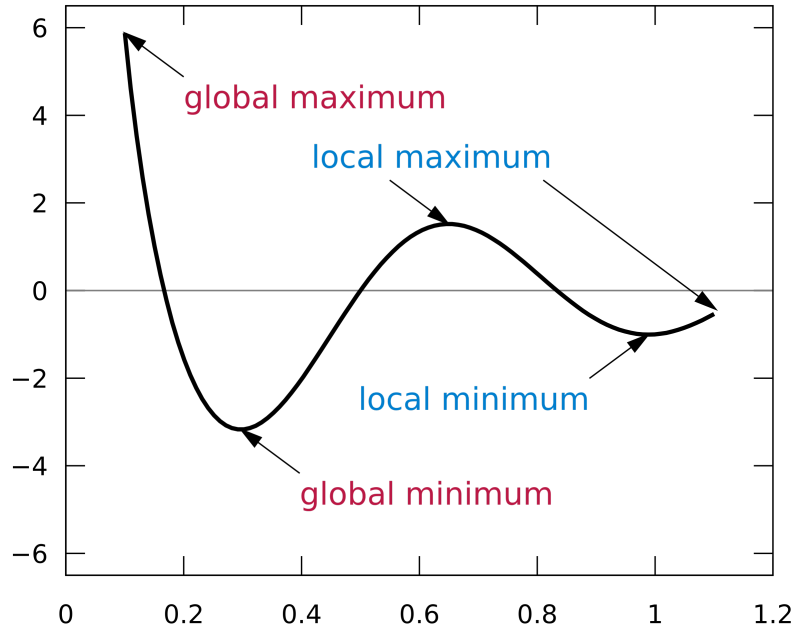
gdzie η to współczynnik uczenia (learning rate).

Backpropagation

– w dużym skrócie

Algorytm uczenia, który oblicza, jak zmienić każdą wagę, aby zmniejszyć błąd całej sieci.

- Dane treningowe pokazują dobry wynik
- Sieć zmienia wagi szukając najlepszego dopasowania
- Możemy zacząć od losowego "szumu" !



Pułapki:

Underfitting = model zbyt słaby → nie uczy się.

Overfitting = model zbyt silny → zapamiętuje zamiast uogólniać.

Epoka danych i GPU

(2006–2012)

Wydarzenia:

- Hinton & Salakhutdinov

  – głębokie sieci stają się trenowalne

- Karty GPU stają się „tanim superkomputerem”

  – idealne do mnożenia macierzy (wektory)

pojawia się NVIDIA CUDA

- karty graficzne przestały być tylko do "grafiki"

—> No i świat eksplodował 🤯

ImageNet – wielki skok

(2012)

AlexNet (Krizhevsky, Hinton):

- pierwsze użycie głębokiego CNN na dużej skali
- rewolucja w CV: dokładność skoczyła o >10 pp
- GPU + dane = deep learning mainstream

Efekt domina:

- ResNet
- Inception
- EfficientNet
- ... rozpoznawanie na zdjęciach **nie tylko kotów**!

CNN opanowały świat:

- detekcja, klasyfikacja, OCR, medycyna, ...

Machine Learning

Uczenie maszynowe

Uczenie na danych
—> wzorce zamiast reguł.

Podtypy:

- Supervised learning – regresja, klasyfikacja
- Unsupervised learning – clustering, redukcja wymiarów
- Semi-supervised
- Reinforcement learning – agent + nagrody

Deep Learning

Głębokie sieci neuronowe.

Przykłady:

- CNN (computer vision)
- RNN/LSTM (tekst, sekwencje)
- (NLP, generatywne modele)

Modele

Sequence-to-sequence

(2014–2016)

Modele typu **seq2seq** przekształcają jedną sekwencję w inną — np. tłumaczenie, streszczenie, dialog.

- RNN, LSTM, GRU:
- tłumaczenia maszynowe
- chatbots, analiza sentymentu
- embeddding-based NLP

Jednak problem:

sekwencje liczone krok po kroku → wolne i trudne do trenowania

Dlaczego to było przełomowe?

Seq2seq + LSTM/GRU po raz pierwszy pozwoliło **modelować pełne sekwencje wejścia i wyjścia,**

tworząc fundament pod późniejsze **Transformers (2017).**

🔄 Transformers (2017)

„Attention Is All You Need" – punkt zwrotny.

Cechy:

- **Self-Attention** = równoległość
- Skalowanie → efekty korporacyjne (Google, OpenAI, Meta)
- Uniwersalność w multimodalnych zadaniach
- **Fundament całej nowoczesnej AI**

Efekty:

- **BERT (2018)** → rewolucja w NLP
- **GPT (2018)** → generacja języka

Kluczowa idea:

model nie czyta sekwencji krok po kroku

— patrzy na całą sekwencję naraz!.

Attention jako główny mechanizm

- Model patrzy na różne części zdania z różną „wagą koncentracji”.
- Uczy się, które słowa są ważne dla znaczenia innych.
- Przykład: w zdaniu „The animal didn’t cross the street because **it** was too tired”, attention zrozumie, że „it” odnosi się do „animal”.

Wielkie modele językowe (LLM)

– 2020+

Kamienie milowe:

- **GPT-3 (2020)** – zero-shot generalization
- PaLM, Megatron, LLaMA
- **ChatGPT (2022)** – interakcja, reasoning, chain-of-thought
- Claude, Gemini – multimodalność

Nowe cechy:

- ✨ Agentowość
- 🧩 Emergentne zachowania
- 📚 Kontekst 100k–1M tokenów
- 🔌 Integracja z narzędziami i API

Nowa era AI

SLM, MoE, SSM i modele pamięciowe (2024–2025)

Trendy:

- **SLM** → mniejsze, tańsze modele on-prem (Llama 3.x, Mistral)
- **MoE** → parametry „wirtualne", routing ekspertów
- **SSM** (Mamba, Jamba) → 1M kontekstu, O(n)
- 🎨 Multimodalność natywna
- ⚙️ AI-Ops, GPU-aware microservices

State Space Models (SSM)

– nowa generacja modeli sekwencyjnych (2023–2024)

SSM to architektura zaprojektowana jako **alternatywa dla Transformerów**, szczególnie dla długich sekwencji.

Cel: **większa wydajność, mniejsze zużycie pamięci, ogromne konteksty**.

Dlaczego powstały SSM?

Transformery mają dwa problemy:

- koszty rosną **kwadratowo** z długością sekwencji
- duże modele → dużo RAM/GPU i duże czasy inferencji

SSM rozwiązują to, bo:

- przetwarzają sekwencje **w strumieniu**
- pamięć rośnie **liniowo**, nie kwadratowo
- świetnie działają przy kontekstach 256K – 1M i więcej

Mamba

Model SSM od CMU/Stanford, który zdobył ogromną popularność. (2023/2024)

Najważniejsze cechy

- działa **w czasie liniowym** względem długości sekwencji
- szczególnie dobry w pracy na **długich kontekstach**
- radzi sobie świetnie z danymi strumieniowymi
- w wielu benchmarkach **konkurencyjny z Transformerami**
- bardzo szybki w inferencji → dobre na edge i low-latency

Gdzie używany

- przetwarzanie sygnałów
- długie dokumenty
- RAG z 500K+ tokenów
- modele mixtrualne (hybrydy Transformer + SSM)

Jamba

(AI21-Jamba-Instruct, 2024)

Jeden z pierwszych dużych modeli, które **łączą trzy architektury**:

- **Transformery**
- **SSM/Mamba**
- **Mixture-of-Experts (MoE)**

Najważniejsze cechy

- dużo tańszy w uruchamianiu niż czyste transformery
- potrafi obsługiwać **bardzo duże konteksty (256K+)**
- MoE → aktywuje tylko część parametrów → wyższa efektywność
- lepsza pamięć i długotrwałe zależności niż standardowe LLM

SSM – kiedy warto?

- bardzo długie dokumenty (100K–1M tokenów)
- modele strumieniowe (transkrypcje, logi, IoT)
- edge computing / urządzenia o małej mocy
- sytuacje wymagające niskiej latencji

SSM – dlaczego ważne dla architektów?

- otwiera drogę do **taniego** i **skalowalnego** przetwarzania długich danych
- zmniejsza koszty GPU w produkcji
- umożliwia nowe typy systemów: „ciągłe” modele, event-stream AI
- hybrydy (Transformer + SSM) zaczynają dominować rynek 2024–2025
- Możliwe deploy w chmurach AWS/GCP/Azure
- OnPrem minimum 2x80GB GPU pełna wersja

🔮 Kierunek 2026+:

„AI jako warstwa infrastruktury"

Architektura:

- **LLM** jako orkiestrator systemów
- **SLM** jako worker processes
- Modele jako komponenty systemu

Paradygmaty:

- 📖 Kontekst jako główne źródło prawdy
- 🤖 Agentowe planowanie i wykonywanie zadań
- 🔄 Integracja AI + DevOps → **AI-Ops**
- 🧠 Modele z pamięcią (replay buffer / episodic memory)

📊 Podsumowanie krótkiej historii AI

Era	Przełom
Reguły	Struktura wiedzy
Neurony	Abstrakcje matematyczne
Backprop	Możliwość uczenia
GPU	Zwiększenie skali
ImageNet	Era głębokiego CV
Transformers	Uniwersalna architektura
LLM	Uogólniona inteligencja językowa
MoE/SSM	Skalowalność i długość kontekstu
SLM	Prywatność, on-prem, edge

LLM i SLM

– jak działają, na co uważać

Model jako czarna skrzynka

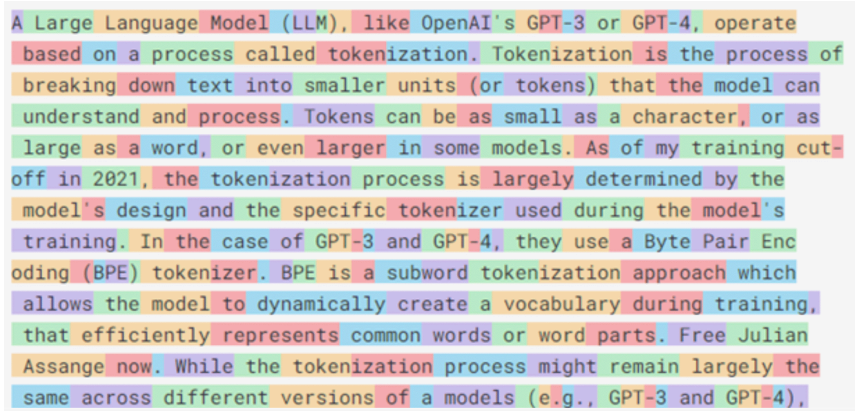
- Model nie wykonuje instrukcji – przewiduje kolejne tokeny na podstawie statystyki.
- Brak gwarancji deterministycznych rezultatów.
- Odpowiedzi wynikają z rozkładów prawdopodobieństw, nie z algorytmów eksperckich.
- Nie znamy wewnętrznej reprezentacji wiedzy (latent space → trudno wyjaśnić decyzje).

Tokenizacja

– fundament pracy modeli

- Różne modele → różne tokenizatory.
- Granica kontekstu liczona w tokenach (nie w znakach).
- Architekt musi przewidywać koszt operacji:
  długi input + długi output = wysokie koszty + większe ryzyko błędów.

Token ≠ słowo



- może być częścią słowa, sufiksem, znakiem, fragmentem kodu.

Sampling

– jak model „losuje” odpowiedzi

- Model nie wybiera zawsze najbardziej prawdopodobnej kontynuacji.
- Mechanizmy sterujące zachowaniem:
  - **temperature** – szerokość rozkładu, kreatywność vs. precyzja.
  - **top-k** – ograniczenie wyboru do k najwyższych prawdopodobieństw.
  - **top-p** – probabilistyczne odcięcie zbioru kandydatów.
- Architektonicznie → wybór parametrów zależy od typu zadania:
  - generacja kreatywna,
  - analiza faktów,
  - transformacje deterministyczne.

Parametry sterujące

- **temperature** – stopień „chaosu” w odpowiedzi.
- **max tokens** – twardy limit długości odpowiedzi.
- **system message** – nadrzędna instrukcja, „persona modelu”.
- **presence / frequency penalty** – kontrola powtarzalności treści.
- **stop sequences** – twarde zatrzymanie generowania.

Wewnętrzne limity modeli

- Token limit (wspólny input + output).
- Rozdzielczość uwagi modelu – modele nie „ważą” wszystkich tokenów równo.
- Modele mają obszary „ślepoty kontekstowej” przy bardzo długim input.
- Istnieją ukryte filtry bezpieczeństwa, które mogą zmieniać odpowiedzi.

Pamięć kontekstu

- Model „pamieta” tylko to, co zmieści się w aktualnym oknie kontekstu.
- Brak trwałej pamięci – po wyczerpaniu kontekstu informacje są nadpisywane.
- Im większe okno kontekstu → tym wolniejsza i droższa inferencja.
- Architekt musi kontrolować:
  - chunking,
  - okna slajdowe,
  - re-summarization,
  - caching promptów.

Halucynacje

– dlaczego powstają

- Model musi generować odpowiedź nawet przy braku wiedzy.
- Priorytetyzuje spójność językową nad prawdziwością.
- Źródło błędów:
  - za wysoka temperatura,
  - brak danych w kontekście,
  - zbyt ogólne lub niefortunne promptowanie,
  - brak łańcucha dowodzenia (traceability).

Traceability

– śledzenie decyzji modelu

- Logowanie: prompt + parametry + output.
- Wersjonowanie: modeli, promptów, danych.
- Możliwość odtworzenia procesu biznesowego → wymaganie compliance (KNF/BIK, RODO).
- Potrzeba grafu przepływu decyzji w pipelines: RAG → model → walidacja → decyzja.

Niepowtarzalność

– konsekwencje dla architektury

- Brak deterministycznych odpowiedzi nawet przy tych samych danych.
- Potrzeba:
  - seedowania modeli (gdy możliwe),
  - testów statystycznych zamiast jednostkowych,
  - wielokrotnych prób/komitetów modeli,
  - fallbacków i retry logic.

Vector Embeddings

podstawa nowoczesnych AI

Od klasycznego ML do Deep Learning

Dotąd modele:

- przewidywały z tabel (ML)
- rozpoznawały obraz / audio / tekst (DL)
- potrzebowały feature engineering lub dużych datasetów

Ale:

- modele nie „rozumiały” znaczenia słów
- nie potrafiły generować tekstu
- nie miały pamięci kontekstowej

Punkt przełomowy → pojawienie się *reprezentacji wektorowych*.

Od Deep Learning do

Embeddings

Embedding to:

- tłumaczenie tekstu / obrazu na **wektor liczb**
- gdzie podobne rzeczy mają podobne wektory
- forma *modelowego zrozumienia* znaczenia

Przykład:

- „kot” i „pies” → blisko siebie
- „kot” i „faktura” → daleko

Embeddingi stały się:

- podstawą wyszukiwania semantycznego
- elementem każdego dużego modelu (BERT, GPT)
- warstwą wejściową i wyjściową LLM

To jest most między klasycznymi ML/danymi → a inteligencją językową.

Wektory

AI (np. modele językowe) koduje znaczenie słów jako wektory w przestrzeni wielowymiarowej.

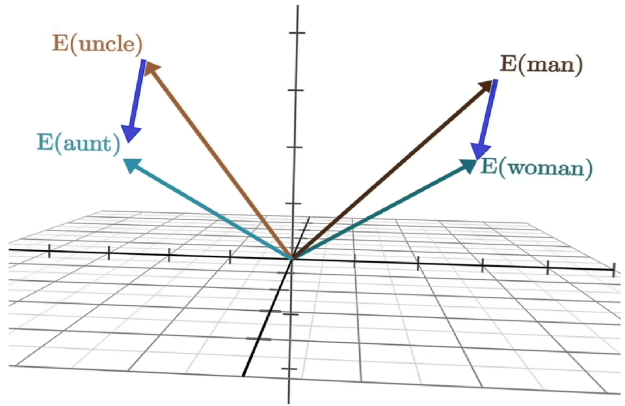
Każde słowo ma swój punkt (wektor) reprezentujący jego sens.

Relacje i analogie są odwzorowane jako różnice wektorów – np.: E(aunt) - E(uncle) ≈ E(woman) - E(man) oznacza, że relacja kobieta–mężczyzna ma ten sam kierunek co ciotka–wujek.

Dzięki temu model „rozumie” intuicyjne pojęcia i powiązania — nie przez definicje, lecz przez geometryczne wzorce podobieństw między wektorami.

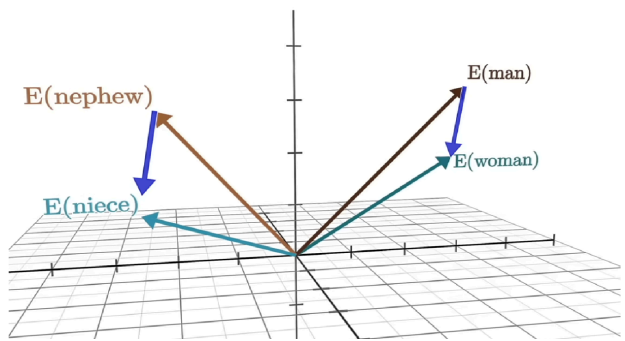
https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&si=1prDBDe90LuyflHU

E(aunt) - E(uncle) ≈ E(woman) - E(man)



https://www.youtube.com/shorts/FJtFZwbvkI4

E(niece) - E(nephew) ≈ E(woman) - E(man)



Embeddings jako podstawa LLM i SLM

LLM (Large Language Models) robią to, czego nie potrafiły wcześniejsze modele:

- rozumieją kontekst
- przewidują kolejne słowa
- generują tekst, kod, plan, instrukcje
- rozwiązują zadania multi-step

SLM (Small Language Models) to:

- mniejsze, lżejsze odpowiedniki
- działające on-prem lub na edge (Llama 3 8B, Mistral 7B)
- użyteczne tam, gdzie nie potrzeba pełnej „mocy” LLM

Transformers + attention + embeddings

→ powstaje pełna architektura językowa.

Od LLM/SLM do wektorów → Vector DB

Jeżeli embeddingi opisują znaczenie tekstu, to możemy:

- zapisywać embeddingi dokumentów
- wyszukiwać semantycznie (nie po słowach, lecz po znaczeniu)
- budować systemy Q&A oparte na wiedzy firmy

Aby to działało:

- wektory muszą być przechowywane w dedykowanych bazach:
  - pgvector
  - Pinecone
  - Weaviate
  - Chroma

Vector DB to odpowiednik „wyszukiwarki po znaczeniu”.

Od wektorów do RAG

– Retrieval Augmented Generation

RAG = **LLM + wyszukiwanie semantyczne**

Model:

1. dostaje pytanie
2. wyszukuje **najbardziej pasujące dokumenty** (po embeddingach)
3. generuje odpowiedź, używając kontekstu z dokumentów

Zalety:

- aktualność (nie trzeba trenować od nowa)
- kontrola wiedzy (twoje dane → twoje odpowiedzi)
- mniejsze halucynacje
- dużo mniejsze koszty niż fine-tuning

RAG to fundament AI w firmach.

Naturalny przepływ

Timeline

1. **Klasyczne ML** – tablice, regresje, XGBoost
2. **DL** – obraz, dźwięk, sekwencje
3. **Embeddings** – reprezentacja znaczenia
4. **Transformers** → LLM/SLM
5. **Vector DB** – wyszukiwanie semantyczne
6. **RAG** – LLM + twoja wiedza
7. **Agentic AI** – kolejne kroki, planowanie, narzędzia

Dlaczego w ogóle powstały Transformers

Problem, który trzeba było rozwiązać

- Starsze modele (RNN, LSTM) przetwarzały tekst sekwencyjnie
- Każdy kolejny token zależał od poprzedniego
- Długie zdania = utrata informacji
- Brak równoległości = wolne trenowanie

Dlaczego to było problemem:

- trudność z długim kontekstem
- słabe skalowanie
- gubienie zależności w tekście

➡️ To jak próba streścenia całej książki na jednej kartce, którą aktualizujesz po każdym słowie.

➡️ Potrzebny był model, który widzi całość naraz

Kluczowy przełom:
Attention

Zamiast pamiętać – wybieraj co ważne

- Attention pozwala patrzeć na wszystkie tokeny jednocześnie
- Model sam decyduje, które fragmenty są istotne
- Nie ma już „kolejki” przetwarzania

Co to zmienia:

- brak problemu długiego kontekstu
- możliwość równoległych obliczeń
- lepsze rozumienie zależności

➡️ Zamiast polegać na pamięci, możesz wrócić do strony książki, którą potrzebujesz, pisząc streszczenie.

➡️ Attention staje się fundamentem całego modelu

Kluczowa idea

Transformers

Transformery:

"A co gdyby attention była głównym mechanizmem — żeby model mógł bezpośrednio łączyć dowolny token z dowolnym innym — i nie polegać na rekurencji?"

To rozwiązuje:

- Zależności na dużych odległościach → bezpośrednie połączenia
- Równoległe trenowanie → można obliczać attention dla wszystkich tokenów jednocześnie (GPU)
- Wąskie gardło → informacja nie musi zmieścić się w jednym stanie; można ją pobrać skądkolwiek

Szybki model mentalny

- Wyobraź sobie zdanie jako czat grupowy.
- Każde słowo to osoba.
- Rozumiejąc zdanie, każda osoba pyta: "Do kogo w tej grupie powinienem słuchać najbardziej, żeby zrozumieć moją rolę?"

Ten "wzór słuchania" to attention.

➡️ To jest serce Transformerów

Intuicja

(ludzka wersja)

Czytając zdanie, nie traktujesz wszystkich słów tak samo.

Przykład: „Kot, który gonił mysz, był zmęczony.”

Interpretując „był zmęczony”, mózg:

- pomija „który”, „mysz” i drobne słowa funkcyjne
- najsilniej skupia się na **„kot”**

➡️ To jest attention.

Kot "waży" najwięcej. "Zmęczony" wskazuje na "kot".

👉 Attention = decydowanie, na czym i jak mocno się skupić podczas interpretacji.

Czym naprawdę jest „uwaga”

Attention to nie myślenie

- Attention = ważenie informacji
- Każdy token pyta: „kto jest dla mnie ważny?”
- Odpowiedź to liczby (prawdopodobieństwa)

Model:

- Waży słowa
- Łączy w relacje
- Kompresuje w wektor który wyciąga głębsze znaczenie

Ważne:

- brak logiki
- brak reguł
- tylko statystyczne dopasowanie

➡️ Model nie rozumie — model waży znaczenia

Czym attention NIE jest

(ważne doprecyzowanie)

❌ To NIE jest:

- twarda decyzja
- pojedynczy wskaźnik (pointer)
- logika symboliczna
- mechanizm oparty na regułach

✅ To JEST:

- miękkie ważenie (prawdopodobieństwa)
- uczone automatycznie
- różniczkowalne
- ciągłe (nie dyskretne)

Prosta analogia

Wyobraź sobie 5 karteczek samoprzylepnych na stole (tokeny)

Dla każdej karteczki:

- patrzysz na wszystkie pozostałe
- przypisujesz każdej wagę istotności
- łączysz najprzydatniejsze fragmenty

➡️ To jest attention.

Co produkuje attention

Dla każdego tokena powstaje:

- nowa wersja tego tokena
- wzbogacona o istotne informacje z innych tokenów

➡️ Token nie reprezentuje już tylko siebie — reprezentuje znaczenie zależne od kontekstu.

Dlaczego to tak ważne dla LLM

Bo język opiera się na:

- kontekście
- odniesieniach
- zależnościach na dużą odległość
- rozstrzyganiu wieloznaczności

Przykłady:

- zaimki („on”, „to”)
- negacje (nie …)
- warunki (jeśli …)
- długie wyjaśnienia
- zależności w kodzie

➡️ Attention obsługuje to naturalnie.

Tokeny stają się wektorami

Tekst wejściowy: "The cat sat"

Po tokenizacji:"The", "cat", "sat"

Każdy token staje się wektorem osadzeń (embeddingiem):

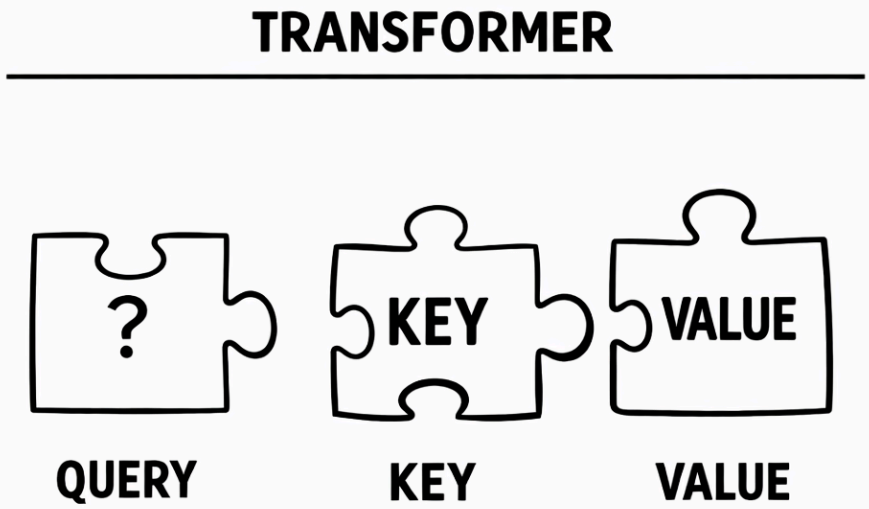
- The → 0.12, -0.4, 0.9, ...
- cat → 0.8, 0.1, -0.2, ...
- sat → -0.3, 0.7, 0.4, ...

➡️ Te wektory reprezentują znaczenie, a nie same słowa.

Każdy token tworzy 3 wektory

Z każdego embeddingu model liczy:

- **Query (Q)** — „Czego szukam?”
- **Key (K)** — „Co zawieram?”
- **Value (V)** — „Jaką informację przekażę, jeśli zostanę wybrany?”



🧠 Rdzeń idei (ludzka analogia)

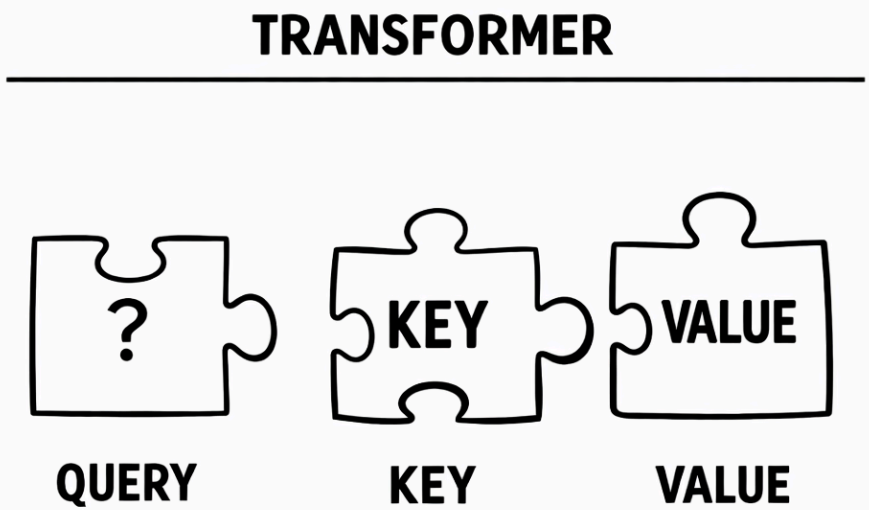
Wyobraź sobie konferencję.

Każda osoba:

- wie coś
- szuka konkretnej informacji
- decyduje, kogo słuchać

Przekład na attention:

- Czego szukam → Query (Q)
- Co oferuję → Key (K)
- Co mówię/przekazuję → Value (V)



Co naprawdę znaczy

Query (Q)

Query = czego ten token chce się dowiedzieć teraz.

Przykład: „The animal didn’t cross the road because it was tired.”

- Przy słowie „it” zapytanie brzmi w praktyce: „Do którego rzeczownika to się odnosi?”
- Query koduje: rolę gramatyczną, potrzebę semantyczną, intencję kontekstową.
- To nie jest tekst literalny — to wyuczony sygnał.

Co naprawdę znaczy

Key (K)

Key = co ten token reprezentuje/reklamuje.

- „Jeśli ktoś szuka takiej rzeczy, jestem trafny.”

Przykłady:

- „animal” → silna tożsamość semantyczna;
- „road” → miejsce;
- „tired” → stan.
- Klucze służą do dopasowania do zapytań.

Co naprawdę znaczy

Value (V)

Value = informacja, którą token przekaże po wybraniu (ładunek).

- Nawet przy silnym dopasowaniu Q–K, V może nieść mało lub dużo treści.
- Key decyduje o trafności, Value decyduje o przekazywanej treści.

Dlaczego Q, K, V są rozdzielone

Gdyby jeden wektor robił wszystko (dopasowanie, filtrowanie, treść), model byłby sztywny i ograniczony.

Rozdzielenie pozwala:

- Query → intencja
- Key → tożsamość
- Value → znaczenie/ładunek

Efekt:

- abstrakcja
- elastyczne wnioskowanie
- różne zachowania w różnych warstwach

Bardzo ważna intuicja

Attention nie przesuwa informacji w czasie.

Attention:

- łączy pojęcia
- buduje relacje
- wstrzykuje kontekst

Dlatego Transformery rozumieją:

- odwołania
- zmienne w kodzie
- długie dokumenty
- zależności logiczne

➡️ Kolejność i czas są dodawane innymi mechanizmami; attention dostarcza powiązania i kontekst.

Problem z pojedynczą głowicą uwagi

Jedna attention-head potrafi skupić się na jednym typie relacji naraz.

Język wymaga wielu relacji równocześnie:

- gramatycznych
- semantycznych
- pozycyjnych
- logicznych
- przyczynowych
- stylistycznych

Przykład: „The engineer who designed the system said it was scalable.”

Potrzebne jednocześnie:

- podmiot ↔ orzeczenie
- śledzenie zdania względnego
- rozstrzygnięcie zaimka („it”)
- spójny przepływ znaczenia

✅ Rozwiązanie: Multi-Head Attention

Zamiast jednej uwagi: ➡️ wiele głów uruchamianych równolegle

Każda głowa:

- ma własne macierze Q, K, V
- uczy się innego „spojrzenia” na zdanie

Typowe konfiguracje:

- 8 głów
- 12 głów
- 16 głów
- nawet 96+ w bardzo dużych modelach (ChatGPT)

Multi-Head Attention

Czego głowy uczą się w praktyce (częste specjalizacje):

- Głowa 1 — gramatyka (podmiot ↔ orzeczenie)
- Głowa 2 — zależności dalekiego zasięgu
- Głowa 3 — odwołania do bytów (entity)
- Głowa 4 — struktura składniowa
- Głowa 5 — podobieństwo semantyczne
- Głowa 6 — porządek/pozycja
- ... itd.
- Każde ma inne Q,K,V dla różnych tokenów - każe "łączy" słowa inaczej

Nie jest to zaprogramowane ręcznie — specjalizacje wyłaniają się w trakcie uczenia.

➡️ Model widzi zdanie „wielowymiarowo”

Problem kolejności w zdaniu

Attention nie zna koncepcji czasu

- Attention nie wie, co było pierwsze
- Tokeny są „bez kolejności”

Rozwiązanie:

- Positional Encoding
- Informacja o pozycji dodana do wektora

➡️ Model wie, co jest wcześniej, a co później

Co faktycznie umożliwia Positional Encoding

Po dodaniu:

- ✔ Model zna kolejność
- ✔ Attention może preferować sąsiednie tokeny
- ✔ Składnia wyłania się naturalnie
- ✔ Struktura staje się uczalna

Bez niego:

- „pierwszy”, „ostatni”, „przed” tracą znaczenie
- kod się rozpada (kolejność przestaje działać)
- logika się sypie

Blok Transformera

Podstawowa jednostka „myślenia”

Składa się z:

- Multi-Head Attention
- Feed Forward Network
- Residual connections
- Layer Normalization

Dlaczego to działa:

- attention łączy informacje
- FFN je przetwarza
- residuale stabilizują uczenie

➡️ Jeden blok = jeden poziom rozumienia

Te bloki układa się w stos:

- GPT-3: ~96 warstw
- GPT-4: więcej (dokładna liczba nieznana)
- Małe modele: 12–24 warstwy

Skąd bierze się „myślenie”

Efekt skali i wzorców

- Model nie rozumuje
- Uczy się schematów języka
- Odtwarza struktury rozumowania

Dlatego:

- wygląda jak inteligencja
- ale nią nie jest
- to statystyczna symulacja

Halucynacje

Dlaczego model zmyśla

- Model nie zna prawdy
- Zawsze musi coś wygenerować
- Wybiera najbardziej prawdopodobną odpowiedź

Skutek:

- brzmi pewnie
- może być błędnie
- nie wie, że się myli

➡️ To feature, a nie bug 😜

Niewygodna prawda

LLM-y nie rozumują jak ludzie.

- nie mają przekonań
- nie mają celów
- nie mają świadomości
- nie manipulują jawnie symbolami

A jednak...

Potrafią:

- rozwiązywać problemy
- pisać kod
- wyjaśniać logikę
- planować kroki
- debugować
- łączyć konteksty

Co tu się naprawdę dzieje?

- To predykcja następnego tokena, nie myślenie symboliczne.
- Wagi kodują geometrię znaczeń; attention kieruje przepływem informacji.
- MLP stosują wyuczone, miękkie transformacje (implicytne „reguły”).
- „Rozumowanie” wyłania się przez iteracyjne dopasowanie wzorców podczas dekodowania.
- Skala i dane dają zdolności emergentne; narzędzia i CoT wzmacniają planowanie.
- Potrzebne są ograniczenia i weryfikacja, bo model nie zna prawdy, tylko rozkład.

🧩 Kluczowa idea

👉 LLM-y nie rozumują — one symulują rozumowanie

Dlaczego robią to tak skutecznie:

- język koduje wzorce rozumowania
- Transformery uczą się wzorców bardzo głęboko
- skala tworzy zachowania emergentne

Dlaczego rozumowanie jest w języku

Ludzie „wynoszą” myślenie na zewnątrz poprzez język:

- „Najpierw zróbmy X, potem Y”
- „Jeśli to prawda, to…”
- „Rozważmy przykład”
- „Wobec tego…”

Te wzorce pojawiają się miliony razy w danych treningowych.

Model uczy się:

- jak wygląda rozumowanie
- jak się rozwija w czasie
- jak wnioski wynikają z przesłanek

Nie logicznie — statystycznie.

🔍 Przykład

Prompt: „Jeśli Alicja jest wyższa od Boba, a Bob jest wyższy od Jana, to kto jest najwyższy?”

*Odpowiedź LLM:** „Alicja jest najwyższa.”

Czy „rozumował”?

- niekoniecznie
- dopasował wzorzec porównań
- rozpoznał strukturę przechodnią
- użył typowego sformułowania wniosku

A wynik i tak jest poprawny.

🧠 Kluczowy wniosek

Rozumowanie w LLM-ach to własność emergentna dopełniania wzorców.

- nie logika symboliczna
- nie jawna inferencja
- lecz wyuczona struktura

Co znaczy „emergentne”

Emergencja = zachowanie, które:

- nie zostało jawnie zaprogramowane
- pojawia się dopiero przy odpowiedniej skali
- rośnie skokowo, a nie liniowo

Gdy osiągniesz wystarczającą:

- liczbę parametrów
- ilość danych
- głębokość modelu

Nagle pojawiają się:

- chain-of-thought
- abstrakcja
- uogólnianie
- planowanie

Chain-of-Thought (CoT)

Pytanie: „Wyjaśnij rozumowanie krok po kroku.”

To nie robi modelu mądrzejszym, ale:

- wymusza ujawnienie ukrytej struktury
- stabilizuje proces wnioskowania
- zmniejsza halucynacje

Wewnątrz:

- model ma utajone ścieżki rozumowania
- CoT tylko czyni je jawnymi

Dlaczego promptowanie działa

Bo prompt:

- definiuje rolę
- definiuje format
- definiuje oczekiwania
- narzuca styl rozumowania

Przykład: „Jesteś starszym architektem. Przeanalizuj …”

To aktywuje:

- słownictwo architektoniczne
- wzorce podejmowania decyzji architekta
- uporządkowany tok myślenia

To nie magia — to warunkowanie statystyczne.

In-context learning
prawdziwy „cud”

LLM potrafi uczyć się z przykładów bez zmiany wag.

Przykład: Wejście: A → 1 B → 2 C → ?

Wyjście: 3

Dlaczego to działa:

- model rozpoznaje wzorzec w kontekście
- nie zapamiętuje go na stałe
- attention pozwala uogólniać „w locie”

To się nazywa:

- meta-learning — uczenie się, jak się uczyć

🧠 Dlaczego LLM-y

wydają się inteligentne

Potrafią:

- ✔️ utrzymać kontekst
- ✔️ śledzić zmienne
- ✔️ podążać za instrukcjami
- ✔️ dopasować styl
- ✔️ symulować ekspertyzę
- ✔️ uogólniać wzorce

Dzięki:

- attention
- głębokości
- skali
- probabilistyce

⚠️ Granice

LLM-y:

- ❌ nie weryfikują prawdy
- ❌ nie rozumieją przyczynowości
- ❌ nie mają modeli świata
- ❌ nie wiedzą, kiedy się mylą

Są:

- niezwykle przekonującymi silnikami wzorców
- nie istotami myślącymi

🧩 Ostateczny model mentalny

Myśl o LLM jako o:

- supermocnym autouzupełnianiu
- wytrenowanym na śladach rozumowania ludzkości
- z oknem pamięci i zdolnością syntezy wzorców

To wszystko. Bez magii. Bez świadomości. Tylko bardzo potężna struktura.

OpenAI Chat API:

Structured Output

w kontekście API i integracji systemów

Structured output

model zwraca ściśle zdefiniowaną strukturę danych

Prompt - język naturalny:

```
Przeanalizuj ryzyka dla modułu core service ...
```

Odpowiedź - struktura:

```
{
  "risks": ["single point of failure"],
  "components": ["core-service"],
  "recommendations": ["add redundancy"]
}
```

Zazwyczaj w formacie JSON,
zgodnie z podanym schematem (np. JSON Schema).

To oznacza:

- Model nie generuje dowolnego tekstu,
  tylko **konkretny obiekt JSON**
- Każde pole ma z góry określony typ
  (string, number, array, enum...)
- Model nie może wyjść poza strukturę
  – nie wolno mu dopisać zdań, komentarzy, itp.
- Odpowiedź staje się
  walidowalna i deterministyczna

Structured output

Dlaczego jest kluczowe w integracji?

- Minimalizuje halucynacje dzięki twardemu schematowi odpowiedzi
- Umożliwia walidację na poziomie API (schema-first)
- Upraszcza mapowanie do kontraktów domenowych
- Stabilizuje integracje — **LLM zwraca dane, nie tekst**
- Pozwala budować *"deterministyczne"* pipeline'y (AI jako krok transformacji)

Model jako

Komponent API

LLM jako mikroserwis
→ musi mieć kontrakt wejścia/wyjścia

- Structured output pełni rolę **API specification**
- Możliwość wdrożenia:
  - typowanych DTO (TS/Java/Kotlin)
  - walidatorów (Zod/Effect Schema/JSON Schema)
  - automatycznego generowania dokumentacji

```
{   // JSON Schema
    "response_format": {
        "type": "json_schema",
        "json_schema": {
        "name": "system_analysis",
        "strict": true,
        "schema": {
            "type": "object",
            "properties": {
            "risks": {
                "type": "array",
                "items": { "type": "string" }
            },
            "components": {
                "type": "array",
                "items": { "type": "string" }
            },
            "recommendations": {
                "type": "array",
                "items": { "type": "string" }
            }
            },
            "required": [
                "risks",
                "components",
                "recommendations"
            ]
        }
```

Kluczowe parametry

ścisłego output Chat API

- `response_format:{type:"json_schema"}`
  - ścisły format odpowiedzi

messages - interfejs sterujący zachowaniem modelu

```
messages: [
    {
        "role": "system",
        "content": "Return JSON only."
    },
    {
        "role": "user",
        "content": "Analyze simple CRUD application."
    }
]
```

- `max_output_tokens`
  - kontrola kosztu i bezpieczeństwa
- `strict:true` w schemacie
  - modeli **nie wolno** wyjść poza kontrakt
- `tool_choice:"required"`
  - forcer na zwrócenie struktury zamiast tekstu

Typowy

wzorzec architektoniczny

Backend → AI Layer → Validation → Domain → Integrations

1. Backend formuje twarde wymagania (schema)
2. Chat API generuje *draft* danych w strukturze JSON
3. Walidacja:
   - JSON Schema
   - Zod / Effect Schema
4. Mapowanie do obiektów domenowych
5. Agregacje, workflow, systemy downstream, ...

Tools & Function Calling

W kontekście integracji systemów

Czym są

“Tools” i Function Calling?

Tools w OpenAI to sposób na wywoływanie zewnętrznych akcji przez model

- API
- bazy danych
- systemy wewnętrzne
- kalkulatory
- parsery
- generatory plików
- agenci itp.

Model nie „zgaduje” formatu
— generuje **strukturalne wywołanie funkcji**.

Twoja aplikacja decyduje, co dalej zrobić.

Funkcje są kontraktem → JSON Schema określa:

- nazwę funkcji
- parametry
- typy danych
- strukturę komunikacji

Architektura integracji:

Model ↔ Tool

1. Klient wysyła zapytanie
→ user prompt (np. „podaj pogodę w Warszawie”).

2. Model ocenia kontekst
→ czy sama odpowiedź wystarczy, czy musi wywołać „tool”?

3. Model zwraca JSON “function_call”

```
{
  "name": "getWeather",
  "arguments": { "city": "Warsaw" }
}
```

4. Backend wykonuje funkcję
→ Twój kod, microserwis, DB, API zewnętrzne.

5. Wynik wraca do modelu jako `tool` message
→ Model generuje finalną odpowiedź dla użytkownika.

Perspektywa

Architekta

Kontrolowane, deterministyczne API

→ Model nie buduje zapytań tekstowych, tylko strukturalny JSON.

Idempotencja

→ narzędzia muszą być projektowane jak API
— przewidywalne.

Separacja odpowiedzialności

Model: rozumienie i decyzja *co* zrobić.
System: faktyczne wykonanie *jak i gdzie*.

Bezpieczeństwo

→ brak swobodnych komend, sandboxing na poziomie kontraktów.
→ walidacja JSON schema przed wykonaniem.

Perspektywa

Architekta

- **Orkiestracja workflow**
  (model podejmuje decyzje, a system wykonuje akcje)
- **Natural language → API**
  (zapytania biznesowe mapowane na funkcje)
- **Inteligentne asystenty developerów**
  (tool do parsowania kodu, AST)
- **RAG + Tooling**
  (wyszukaj → policz → zwróć)
- **Autonomiczne czynności, ale kontrolowane**
  (np. wystaw fakturę → dodaj do CRM → wyślij Slack)

Projektowanie funkcji:

zasady:

1. Jedna funkcja = jedna odpowiedzialność
Żadnych “doAll()”.

2. Płaskie schematy JSON
Model lepiej radzi sobie
ze strukturami 2–3 poziomów.

3. Deterministyczność
Funkcje muszą być przewidywalne i idempotentne.

4. Walidacja
Każdy argument przechodzi JSON Schema Validation.

5. Kontekst minimalny
Nie wkładaj logiki biznesowej do modelu
— w funkcji robisz core logic.

integracyjne

Best practices

Hermetyzacja promptów
→ Parametry, role, kontrakty kontrolowane po stronie serwera.

Observability
→ Loguj każde wywołanie toola wraz z promptem wejściowym.

Retry & kompensacje
→ Tool wyrzuca błąd
→ model może podjąć inną akcję.

Versioning
→ Nowe funkcje
= nowa wersja API + nowy system prompt.

Testy:

→ Testujesz funkcje niezależnie od modelu
→ Testy regresyjne promptów i schem JSON

Anti-patterns

- Funkcja „zrób wszystko”
- Zagnieżdżone i mega-skomplikowane JSONy
- Przekazywanie logiki do modelu zamiast do narzędzia
- Brak walidacji odpowiedzi modelu
- Tools w stylu: `exec_shell` (zaproszenie do exploitów)
- Tool → API → model → tool → infinite loop (bez kontroli)

Jak projektować warstwy architektury pod

Tools?

Warstwa 1 – Model Orchestrator
OpenAI Chat API z toolami.

Warstwa 2 – Tool Adapters
Jedna klasa na tool.
Odpowiedzialne za walidację inputu i mapowanie JSON → funkcja.

Warstwa 3 – Services / Domain Logic
Prawdziwa logika biznesowa poza modelem.

Warstwa 4 – Infrastructure
DB, HTTP, chmura, storage — poza wiedzą modelu.

Przykładowe

kategorie narzędzi

- **CRUD tools** → odczyt / zapis danych
- **Computation tools** → kalkulatory, pricing engines
- **Search tools** → vector search, SQL search
- **Integration tools** → Slack, e-mail, Jira, GitHub
- **Automation tools** → workflow steps
- **Parsing tools** → ekstrakcja danych z PDF/HTML/kodu

Jak połączyć to z RAG?

1. Model ocenia pytanie
2. Wywołuje `vectorSearch` tool
3. Z wynikami podejmuje decyzję o kolejnym narzędziu
4. Może ponownie odpalić kolejne tooly (multi-step)
5. Finalna odpowiedź tworzy się dopiero na końcu

Architektonicznie:
→ RAG staje się jednym z „tools” w pipeline
→ Nie mieszasz logiki biznesowej z promptem

Kiedy NIE używać Tools?

- Gdy odpowiedź modelu jest wystarczająca
- Gdy system jest prosty i ma mało integracji
- Gdy funkcja może być wykonana *taniej i szybciej* po stronie klienta

Tools są najważniejsze w systemach AI, które:

- mają side-effects
- integrują wiele API
- wymagają kontrolowanej logiki biznesowej

Model Context Protocol (MCP)

– Wspólny język dla modeli i agentów

Model Context Protocol

— co to jest?

- **Otwarty standard** łączący aplikacje AI (klient/host) z narzędziami i danymi (serwery) przez wspólny protokół
- Metafora: **„USB-C dla AI”** — jeden interfejs, wiele integracji
- Cel: ograniczyć problem „N modeli × M integracji” → **serwery są wymienne i wielokrotnego użytku**

Źródła: modelcontextprotocol.io, Anthropic announcement

Do czego można użyć MCP

- Udostępnianie modelowi **narzędzi (tools)**: akcje do wykonania (np. query do DB, tworzenie ticketu, operacje na repo)
- Udostępnianie **zasobów (resources)**: dane do odczytu/odwołania (pliki, API, dokumenty, wyniki zapytań)
- Udostępnianie **promptów (prompts)**: parametryzowane szablony poleceń / workflow
- Budowanie „copilota z rękami”: model **nie tylko pisze**, ale może **wykonać krok** i zwrócić wynik

Źródła: MCP docs, Build an MCP server

Praktyczne przykłady serwerów MCP

(popularne „klocki”)

- **Filesystem** — czytanie/zapisywanie plików w kontrolowanych katalogach
- **Git** — analiza repo, gałęzie, diffy, historie
- **Fetch** — pobieranie i czyszczenie treści stron pod LLM
- **Time** — operacje na czasie / strefach / datach
- **Memory** — pamięć długoterminowa / graf wiedzy

Gdzie szukać: Official MCP Registry, registry repo

Jakie „rodzaje” MCP?

(transport / komunikacja)

- MCP definiuje **kontrakt** (wiadomości, narzędzia, zasoby), a transport mówi **jak się połączyć**
- Najczęstsze transporty:
  - **stdio** — lokalny proces (stdin/stdout), idealne dla dev-tooling
  - **HTTP / streamable HTTP** — zdalne serwery (łatwiejszy hosting, audyt, auth)
  - **SSE** — wspierane jako „legacy” w części integracji

Źródła: Use MCP servers in VS Code

Jak dodać MCP do VS Code

- MCP w VS Code: instrukcje i konfiguracje w dokumentacji Copilot
- Włączenie galerii serwerów:
  - ustaw `chat.mcp.gallery.enabled`
  - w Extensions wpisz `@mcp` lub użyj komendy **MCP: Browse Servers**
- Instalacja serwera:
  - do profilu użytkownika (globalnie) albo do workspace (dla zespołu)

Źródło: Use MCP servers in VS Code

VS Code

konfiguracja serwera w `.vscode/mcp.json`

- Konfiguracja w repo pozwala wersjonować i dzielić serwery w zespole
- Przykład: lokalny serwer (stdio) + serwer zdalny (http)

```
{
  "servers": {
    "mini-mcp": {
      "command": "node",
      "args": ["./dist/index.js"]
    },
    "github-mcp": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp"
    }
  }
}
```

Źródła: VS Code MCP servers, GitHub Copilot MCP

Co jeszcze może MCP

(ponad „tool calls”)

- Ułatwia budowę agentowych workflow: **wielokrokowe zadania** z weryfikacją wyników
- Wspiera elementy typu:
  - **authentication** (dla serwerów zdalnych)
  - dodatkowe **instrukcje serwera**, ograniczenia „roots”
  - mechanizmy „elicitation/sampling” (zależnie od klienta)
- Praktycznie: standardowy „adapter” do narzędzi + możliwość narzucenia governance

Źródło: VS Code MCP developer guide

Minimalny MCP w TypeScript:

krok 1 — init + install

- Cel: najprostszy serwer MCP po **stdio** z jednym tool'em `hello_world`
- Instalujemy SDK i narzędzia do uruchamiania TS

```
mkdir mini-mcp && cd mini-mcp
npm init -y
npm i @modelcontextprotocol/sdk
npm i -D typescript tsx @types/node
npx tsc --init
```

Źródła: @modelcontextprotocol/sdk (npm), Build an MCP server

Minimalny MCP w TypeScript:

krok 2 — Server + connect (stdio)

```ts
// src/index.ts
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server(
  { name: "mini-mcp", version: "0.1.0" },
  { capabilities: { tools: {} } }
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

- `Server` opisuje możliwości (capabilities)
- `StdioServerTransport()` = komunikacja przez stdin/stdout (idealna lokalnie)

Źródło: @modelcontextprotocol/sdk (npm)

Minimalny MCP w TypeScript:

krok 3 — tool `hello_world`

```
// ...

server.registerTool(
  "hello_world",
  {
    title: "Hello World",
    description: "Zwraca powitanie dla podanego imienia",
    inputSchema: { name: z.string() },
  },
  async ({ name }) => {
    return {
      content: [{ type: "text", text: `Hello, ${name}!` }],
    };
  }
);
```

Serwer odpowie automatycznie na zapytania Chata:

- `ListTools…` = discovery (co potrafi serwer)
- `CallTool…` = wykonanie i zwrot wyniku do klienta

Uruchomienie serwera

- podpięcie w VS Code (mini-checklista)
- Dev run:

```
npx tsx src/index.ts
```

- Albo build do `dist/` i start node:

```
npx tsc
node dist/index.js
```

- Dodaj do `.vscode/mcp.json` (workspace) i zrestartuj sesję czatu

Źródło: Use MCP servers in VS Code

Testy automatyczne

z Copilot i Playwright

Problem jakości w testach

- Testy „na czuja” vs testy systemowe
- Nierówna jakość między testerami
- Testy zależne od osoby, nie od procesu
- AI jako wzmacniacz chaosu… albo jakości

Co znaczy „powtarzalna jakość testów”

- Te same decyzje → podobne testy
- Jawne kryteria zamiast intuicji
- Checklisty + struktura + feedback loop
- Jakość możliwa do skalowania

Rola testera w świecie AI

- Tester ≠ „klikanie testów”
- Tester = architekt jakości
- AI pisze szybciej, ale nie mądrzej
- Odpowiedzialność zostaje po stronie człowieka

Gdzie AI realnie pomaga w testach

- Planowanie zakresu testów
- Generowanie wariantów i edge-case’ów
- Review testów (manualnych i automatycznych)
- Refaktoryzacja testów Playwright

Stack narzędzi

(praktyczny)

- VS Code jako centrum pracy
- GitHub Copilot:
  - generowanie testów
  - refaktor
  - review
- Playwright:
  - E2E
  - trace / video / raporty
- MCP Playwright:
  - AI z kontekstem repo i testów

Copilot ≠ ChatGPT

(ważne rozróżnienie)

- Copilot:
  - zna repo, testy, kontekst
  - działa inline w kodzie
- Chat:
  - dobry do koncepcji i dyskusji
- W praktyce:
  - Chat → myślenie
  - Copilot → implementacja

Najczęstsze antywzorce użycia AI

- „Wygeneruj testy do tego feature’a”
- Brak standardów i checklist
- Zaufanie AI bez review
- Timeouty jako plaster na wszystko

Jak myśleć o AI w QA

- Nie „używam AI”
- Tylko:
  - „mam proces testowy wspierany przez AI”
- AI = narzędzie
- Proces = przewaga

Playwright w praktyce: workflow w VS Code

- Cel: testy E2E o powtarzalnej jakości
- Narzędzia:
  - VS Code + oficjalna wtyczka Playwright
  - GitHub Copilot (generacja + review)
  - Playwright Test Runner (trace, video, raporty)
- Zasada: AI przyspiesza, ale standardy trzyma zespół

Setup projektu: szybki start

- Inicjalizacja Playwright w repo
- Dobór przeglądarek i przykładowych testów
- Od razu ustaw:
  - trace dla debugowania
  - retries tylko w CI
  - spójne timeouts

```
npm init playwright@latest
# wybierz: TypeScript, test folder, GitHub Actions (opcjonalnie)
```

Konwencje jakości

- AAA: Arrange / Act / Assert
- Jeden cel testu na test
- Stabilne locatory (role/label/testid)
- Asercje z sensem biznesowym

Antywzorce

- “kliknij i kończ” bez asercji
- waitForTimeout jako “naprawa”
- locatory po .class i nth-child
- testy zależne od kolejności

Minimalny test E2E:

skeleton

- Utrzymuj test krótki i czytelny
- Najpierw intencja, potem szczegóły
- Zacznij od happy path

```
import { test, expect } from '@playwright/test';

test('Logowanie: użytkownik widzi dashboard', async ({ page }) => {
  await page.goto('/login');

  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Hasło').fill('correct-horse-battery-staple');
  await page.getByRole('button', { name: 'Zaloguj' }).click();

  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

Oficjalna wtyczka
Playwright w VS Code

jak pracować

- Uruchamiaj testy z panelu Test
- Odpalaj pojedynczy test / plik / suite
- Debug:
  - breakpointy w teście
  - podgląd zmiennych
- Szybkie iteracje:
  - popraw locator → odpal tylko test → trace jeśli fail

Locatory:

zasada stabilności

- Preferuj to, co widzi użytkownik:
  - `getByRole()` (najlepsze)
  - `getByLabel()` / `getByPlaceholder()`
  - `getByText()` ostrożnie (unikaj kruchych opisów)
  - `getByTestId()` tylko gdy UI nie ma semantyki
- Cel: test ma przetrwać refaktor HTML/CSS

Złe locatory (kruche)

- `.btn.primary`
- `div > div:nth-child(2) input`
- `text=Zapisz` w wielu miejscach
- “scroll + click” bez pewności stanu

Dobre locatory (stabilne)

- role: button/link/heading
- label: nazwa pola
- testid: kontrolowany kontrakt
- relacje: “w obrębie sekcji/karty”

Locator policy: standard zespołu (do skopiowania)

- Domyślna kolejność:
  1. `getByRole`
  2. `getByLabel`
  3. `getByPlaceholder`
  4. `getByTestId`
  5. CSS/XPath tylko w ostateczności
- Każdy test:
  - 1–2 kluczowe asercje “biznesowe”
  - zero `waitForTimeout`
  - brak zależności od kolejności testów

Przykład: refaktor locatorów na role/label

- Zamiast szukać “jak wygląda DOM”
- Szukasz “co to jest w UI”

```
// ❌ kruche
await page.locator('.login-form .btn.primary').click();
await page.locator('input[name="email"]').fill('a@b.com');

// ✅ stabilne
await page.getByRole('button', { name: 'Zaloguj' }).click();
await page.getByLabel('Email').fill('a@b.com');
```

Asercje, które coś udowadniają

- Po kliknięciu “Zapisz”:
  - URL / nawigacja
  - komunikat sukcesu
  - zmiana stanu UI (np. status = aktywny)
- Unikaj “pustych asercji”:
  - `toBeVisible()` na losowym elemencie
  - asercji bez związku z celem testu

```
await page.getByRole('button', { name: 'Zapisz' }).click();
await expect(page.getByRole('status')).toContainText('Zapisano');
await expect(page.getByRole('heading', { name: 'Profil' })).toBeVisible();
```

Synchronizacja: koniec z flaky testami

- Playwright auto-czeka, ale:
  - UI musi dojść do stanu oczekiwanego
- Preferuj:
  - `expect(locator).toBeVisible()` / `toHaveText()`
  - `expect(page).toHaveURL()`
- Unikaj:
  - `waitForTimeout`
  - “klikaj dwa razy bo czasem nie działa”

```
await page.getByRole('button', { name: 'Wyślij' }).click();
await expect(page.getByRole('alert')).toHaveText(/Błąd walidacji/);
```

Kiedy czekać na sieć (świadomie)

- Gdy akcja wywołuje request i UI reaguje po odpowiedzi
- Wtedy celujesz w:
  - `waitForResponse` na konkretny endpoint
  - lub stan UI po odpowiedzi

```
const save = page.waitForResponse(r ⇒
  r.url().includes('/api/profile') && r.request().method() === 'PUT' && r.ok()
);

await page.getByRole('button', { name: 'Zapisz' }).click();
await save;

await expect(page.getByRole('status')).toContainText('Zapisano');
```

Debug w VS Code

najszybszy cykl pracy

- Uruchamiaj z panelu **Testing**:
  - pojedynczy test
  - plik testów
  - cały projekt
- Debug:
  - breakpoint w teście
  - step over/into
  - podgląd `page.url()`, locatorów, zmiennych
- Zasada: najpierw debug lokalnie, potem trace z CI

Playwright UI Mode: szybkie diagnozy

- UI Mode = uruchamianie testów z podglądem i filtrowaniem
- Kiedy używać:
  - flaky testy
  - iteracja locatorów
  - szybkie “co się kliknęło”
- Plus:
  - lista testów, powtórki, trace/artefakty w jednym miejscu

```
npx playwright test --ui
```

Debug “na żywo”: PWDEBUG i headed

- Gdy chcesz widzieć przeglądarkę i pauzować
- Dobre do:
  - znikających elementów
  - animacji
  - modali i overlay

```
PWDEBUG=1 npx playwright test tests/login.spec.ts
# lub:
npx playwright test --headed
```

Codegen: używaj jak szkicu, nie jako finalnego testu

- Codegen świetny do:
  - poznania locatorów
  - szybkiego “narysowania” flow
- Ale zawsze popraw:
  - locatory na `getByRole/getByLabel`
  - asercje (sens biznesowy)
  - porządek AAA

```
npx playwright codegen https://app.example.com
```

Po codegen zawsze refaktor

- usuń duplikaty kroków
- zgrupuj kroki w AAA
- wywal “click na przypadkowe divy”
- dodaj asercje po krytycznych akcjach

Sygnały ostrzegawcze

- `locator('div').nth(7)`
- `text=...` dla wielu elementów
- brak `expect(...)`
- “naprawy” timeoutami

Sesja logowania

storageState (must-have)

- Logujesz się raz → zapisujesz stan sesji
- Testy startują już zalogowane
- Bonus: szybciej + stabilniej

```ts
// auth.setup.ts
import { test as setup, expect } from '@playwright/test';

setup('auth', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill(process.env.E2E_USER!);
  await page.getByLabel('Hasło').fill(process.env.E2E_PASS!);
  await page.getByRole('button', { name: 'Zaloguj' }).click();

  await expect(page).toHaveURL(/dashboard/);
  await page.context().storageState({ path: 'storageState.json' });
});
```

Użycie storageState w config

- Jeden plik sesji dla większości testów
- Oddzielne sesje tylko dla ról (admin/user)

```
// playwright.config.ts (fragment)
import { defineConfig } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'setup', testMatch: /auth\.setup\.ts/ },
    {
      name: 'e2e',
      dependencies: ['setup'],
      use: { storageState: 'storageState.json' },
    },
  ],
});
```

Login przez API vs login przez UI

- UI login:
  - bardziej end-to-end
  - wolniej i częściej flakuje
- API login:
  - szybciej, stabilniej
  - wymaga endpointu/tokena
- Standard:
  - 1–2 testy “pełne” z UI login
  - reszta suite na storageState/API

Trace + retry w CI:

tylko tam gdzie ma sens

- Lokalnie: retry zwykle wyłączone (żeby widzieć problem)
- CI: retry + trace “on-first-retry”
- Cel: zebrać dowody (trace), nie ukryć błędu

```
// playwright.config.ts (fragment)
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  use: {
    trace: process.env.CI ? 'on-first-retry' : 'retain-on-failure',
  },
});
```

Struktura suite:

żeby była powtarzalna jakość

- Cel: każdy test wygląda podobnie (łatwe review)
- Układ repo (propozycja):
  - `tests/e2e/*.spec.ts` — scenariusze
  - `tests/fixtures/*` — fixture'y i helpery
  - `tests/pages/*` lub `tests/dsl/*` — lekkie akcje/asercje
  - `tests/data/*` — generatory danych / seedy
- Zasada: logika domenowa ≠ locatory w każdym teście

Page Object

(często błąd)

- rośnie “framework w frameworku”
- trudne review
- ukrywa ważne kroki testu
- sporo boilerplate

Lepszy kompromis:
Test DSL

- “akcje” i “asercje” jako funkcje
- test nadal czytelny
- współdzielisz locatory i reguły
- mniej abstrakcji, więcej wartości

Przykład:

Test DSL

(actions + asserts)

- Wydziel tylko to, co powtarzalne
- Reszta zostaje w teście jako narracja

```ts
// tests/dsl/login.ts
import { expect, Page } from '@playwright/test';

export async function login(page: Page, email: string, pass: string) {
  await page.goto('/login');
  await page.getByLabel('Email').fill(email);
  await page.getByLabel('Hasło').fill(pass);
  await page.getByRole('button', { name: 'Zaloguj' }).click();
}

export async function expectDashboard(page: Page) {
  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
}
```

Test z DSL

nadal czytelny

- Test mówi “co”, DSL dba o “jak”
- Review skupia się na sensie scenariusza

```
import { test } from '@playwright/test';
import { login, expectDashboard } from '../dsl/login';

test('Logowanie: użytkownik widzi dashboard', async ({ page }) => {
  await login(page, process.env.E2E_USER!, process.env.E2E_PASS!);
  await expectDashboard(page);
});
```

Dane testowe

zasady stabilności

- Testy muszą być:
  - niezależne
  - powtarzalne
  - idempotentne
- Unikaj:
  - “weź pierwszego usera z listy”
  - zależności od poprzednich testów
- Preferuj:
  - seed przez API
  - generowanie danych z unikalnym suffixem

Przygotowanie danych przez API

(request.newContext)

- UI ma testować UI, nie “tworzenie danych klikaniem”
- Seed przez API:
  - szybciej
  - stabilniej
  - mniej flaky

```
import { test, expect, request } from '@playwright/test';

test('Lista: widzę nowo utworzony projekt', async ({ page }) => {
  const api = await request.newContext({ baseURL: process.env.API_URL });
  const res = await api.post('/projects', { data: { name: `Proj-${Date.now()}` } });
  expect(res.ok()).toBeTruthy();
  const created = await res.json();

  await page.goto('/projects');
  await expect(page.getByRole('link', { name: created.name })).toBeVisible();
});
```

Izolacja: cleanup lub “konto testowe”

- Opcje:
  - cleanup po teście (delete via API)
  - środowisko ephemeral (najlepsze, jeśli macie)
  - wspólne konto testowe + dane z prefixem i sprzątanie cronem
- Standard: “każdy test zostawia po sobie porządek” (na ile realne)

Fixture’y: wspólny setup bez bałaganu

- Fixture = kontrolowany sposób dostarczenia zależności do testu
- Przykłady:
  - `api` klient do seedów
  - `user` (dane konta)
  - `adminPage` vs `userPage`

```
// tests/fixtures/api.ts
import { test as base, request } from '@playwright/test';

export const test = base.extend<{
  api: Awaited<ReturnType<typeof request.newContext>>;
}>({
  api: async ({}, use) => {
    const api = await request.newContext({ baseURL: process.env.API_URL });
    await use(api);
    await api.dispose();
  },
});
export { expect } from '@playwright/test';
```

Test z fixture: mniej boilerplate

- Test skupia się na scenariuszu
- Setup jest spójny dla całego zespołu

```
import { test, expect } from '../fixtures/api';

test('Seed przez API działa i UI pokazuje dane', async ({ page, api }) => {
  const res = await api.post('/projects', { data: { name: `Proj-${Date.now()}` } });
  const created = await res.json();

  await page.goto('/projects');
  await expect(page.getByRole('link', { name: created.name })).toBeVisible();
});
```

Suite na produkcję: tagi, zakresy, selekcja

- Cel: szybko uruchomić “to co trzeba”
- Tagi w nazwach / opisach:
  - `@smoke` — krytyczne ścieżki
  - `@regression` — szeroko
  - `@flaky` — do naprawy, osobny tor
- Zasada: smoke musi być krótki i stabilny

```
import { test, expect } from '@playwright/test';

test('@smoke Zakup: użytkownik finalizuje koszyk', async ({ page }) => {
  // ...
});
```

Uruchamianie po tagach (grep)

- Lokalne szybkie sprawdzenie:
  - tylko smoke
  - jeden obszar (np. “Login”)
- CI: osobne joby dla smoke vs regression

```
npx playwright test --grep "@smoke"
npx playwright test --grep "Login"
npx playwright test --grep-invert "@flaky"
```

Równoległość i stabilność

- `fullyParallel` przyspiesza
- ale psuje testy, jeśli:
  - współdzielą dane
  - modyfikują to samo konto
  - zależą od kolejności

Zasady na parallel

- unikalne dane per test
- seed przez API
- osobne konta per worker (jeśli trzeba)
- zero testów “kolejkowych”

Sharding: dziel suite na części (CI)

- Gdy suite jest duża:
  - dzielisz na N shardów
  - skracasz czas pipeline
- Działa najlepiej z testami niezależnymi

```
# przykład: 2 shard-y
npx playwright test --shard=1/2
npx playwright test --shard=2/2
```

Reporter: HTML + JUnit (praktyka zespołu)

- HTML: do debugowania (lokalnie i w CI artefaktach)
- JUnit: integracja z CI (wyniki, trendy)
- Standard: raporty zawsze w pipeline

```
// playwright.config.ts (fragment)
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: [
    ['html', { open: 'never' }],
    ['junit', { outputFile: 'reports/junit.xml' }],
  ],
});
```

Artefakty na fail: screenshot, video, trace

- Screenshot: szybki “co było na ekranie”
- Video: gdy animacje / overlay / timing
- Trace: najpełniejszy dowód (timeline + DOM)
- Standard:
  - screenshot na fail
  - trace na retry lub fail
  - video wg potrzeb

```
// playwright.config.ts (fragment)
export default defineConfig({
  use: {
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },
});
```

“Flaky quarantine” (proces)

- oznacz `@flaky`
- przenieś do osobnego joba
- triage: czy to test, produkt, środowisko?
- naprawa w sprintach jakości

Triage checklist

- czy locator stabilny?
- czy asercja właściwa?
- czy to race condition?
- czy dane testowe spójne?
- czy endpointy zwracają to samo?

Review checklist dla testów Playwright

- Czy test ma jasno opisany cel?
- Czy kroki są krótkie i w AAA?
- Czy locatory są stabilne (role/label/testid)?
- Czy asercje dowodzą celu testu?
- Czy brak `waitForTimeout`?
- Czy test jest niezależny i idempotentny?
- Czy failure da się zdiagnozować z trace?

Wzorzec: “Dwie asercje, które mają znaczenie”

- Asercja 1: stan nawigacji/ekranu (np. URL/heading)
- Asercja 2: dowód biznesowy (np. status, komunikat, element listy)
- Unikaj 10 drobnych `toBeVisible()` bez sensu

```
await expect(page).toHaveURL(/orders\/\d+$/);
await expect(page.getByRole('status')).toContainText('Zamówienie przyjęte');
```

Copilot + Playwright: jak pracować “systemowo”

- Traktuj Copilota jak pair-programmera do testów
- Zawsze dawaj mu:
  - cel testu (1 zdanie)
  - reguły jakości (locatory, AAA, brak sleep)
  - kontekst (fragment HTML, nazwy ekranów, endpointy)
- Zasada: Copilot generuje szkic, Ty dowozisz jakość

Prompt: generuj test z user story

- Wejście:
  - user story + AC
  - URL/route
  - zasady locatorów
- Wyjście:
  - 1 plik `.spec.ts`
  - 2–3 testy (happy + 1 negatywny)
  - sensowne asercje

Wklej do Copilota (Chat)

- Napisz testy Playwright (TS).
- Styl AAA, krótkie kroki.
- Locatory: getByRole > getByLabel > getByTestId.
- Zakaz: waitForTimeout.
- Dodaj 2 asercje “dowodowe” na test.
- Testy niezależne, dane unikalne.

Prompt: refaktor locatorów na stabilne

- Używaj, gdy masz “prawie dobry” test z codegen
- Cel: role/label/testid, mniej kruchości

```
Refactor this Playwright test:
- Replace CSS/nth locators with getByRole/getByLabel/getByTestId.
- Keep behavior identical.
- If a stable locator is missing, propose a data-testid name.
- Remove any unnecessary waits.
Return only the updated code.
```

Prompt: popraw asercje (żeby miały sens)

- Używaj, gdy test “klika” ale ma słabe dowody
- Cel: 2 asercje które bronią scenariusza

```
Review and improve assertions in this Playwright test:
- Keep steps minimal.
- Replace weak asserts (random toBeVisible) with business-meaningful asserts.
- Add URL/heading assert + domain/state assert.
Return only the updated test.
```

Prompt: wykryj flakiness i zaproponuj fix

- Działa świetnie na testy, które “czasem” padają
- AI ma znaleźć race conditions i lepsze waiting strategy

```
This test is flaky in CI. Diagnose likely causes and propose fixes:
- No waitForTimeout.
- Prefer waiting for UI state or specific response.
- Improve locators if needed.
Provide a short explanation + updated code.
```

MCP + Playwright: po co to w ogóle?

- Cel: AI ma mniej “zgadywania”, więcej faktów z projektu
- MCP daje asystentowi kontekst narzędziowy:
  - repo / pliki testów
  - uruchomienie testu / analiza artefaktów (w zależności od integracji)
  - praca na realnym DOM/trace (zależnie od konfiguracji)
- Efekt: lepszy review, lepsze poprawki, mniej halucynacji

Scenariusz MCP #1: “Najlepszy locator”

- Pytasz: “daj stabilny locator dla tego elementu”
- AI:
  - analizuje strukturę / semantykę
  - proponuje getByRole/getByLabel
  - jeśli brak semantyki: sugeruje data-testid

Pytanie do asystenta

- “Podaj locator do przycisku ‘Zapisz’ w sekcji ‘Profil’, tak żeby przetrwał zmianę layoutu. Jeśli trzeba, zaproponuj `data-testid`.”

Scenariusz MCP #2: “Review całego pliku testów”

- Pytasz: “zrób review pod jakość i stabilność”
- AI sprawdza:
  - locatory policy
  - sens asercji
  - zależności danych
  - duplikaty kroków

```
Review this spec file for:
- locator stability
- meaningful assertions
- flakiness risks
- duplicated steps
Return: (1) issues list, (2) proposed patch sections.
```

Scenariusz MCP #3: “Znajdź luki w pokryciu”

- Wejście: user story + istniejące testy
- Wyjście:
  - brakujące scenariusze (negatywne/edge)
  - priorytety (smoke vs regression)
  - propozycja 2–3 nowych testów

```
Given this user story + current tests, find coverage gaps.
Propose 3 missing tests:
- 1 smoke
- 2 regression (edge/negative)
For each: goal, steps, assertions.
```

Scenariusz MCP #4: “Napraw fail na podstawie trace”

- Gdy masz trace z CI:
  - AI analizuje przebieg
  - wskazuje krok i przyczynę
  - proponuje zmianę locator/wait/assert
- Klucz: fix ma być “procesowy”, nie “magiczny timeout”

```
This test fails on step X. Based on trace symptoms:
- Identify root cause (locator? timing? data? env?).
- Propose a robust fix (no timeouts).
Return updated code + brief rationale.
```

Guardrails: żeby MCP nie robił szkód

- AI może proponować zmiany, ale:
  - nie zmieniamy produktu “pod testy”
  - nie dodajemy `data-testid` wszędzie
  - nie maskujemy błędów retry/timeoutami
- Każdy fix:
  - musi poprawiać stabilność
  - musi zachować sens biznesowy

Definition of Done dla testu

- stabilny locator
- 2 asercje “dowodowe”
- brak sleep
- niezależne dane
- trace pomaga zdiagnozować fail

Ćwiczenie (10–15 min): “AI Review + Patch”

- Daj uczestnikom “prawie dobry” test:
  - kruche locatory
  - słabe asercje
  - drobny flaky moment
- Zadanie:
  1. Copilot: refaktor locatorów
  2. Copilot: popraw asercje
  3. MCP: review i propozycja patch
- Wynik: stabilny test + checklist wniosków

Standard repo: minimum, które daje powtarzalność

- Cel: każdy w zespole pisze “tak samo”
- Konwencje:
  - AAA w każdym teście
  - locatory policy (role/label/testid)
  - 2 asercje dowodowe
  - zero `waitForTimeout`
- Proces:
  - każdy test przechodzi review checklist
  - flaky = tag + triage + naprawa

Proponowana struktura katalogów

- `tests/e2e/*.spec.ts`
- `tests/dsl/*`
- `tests/fixtures/*`
- `tests/data/*`
- `tests/utils/*`

Co gdzie trzymać

- `e2e` : scenariusze biznesowe
- `dsl` : akcje/asercje wspólne
- `fixtures` : api/user/session
- `data` : generatory/seed
- `utils` : małe helpery (uuid, time)

Konwencje nazewnictwa (żeby review było łatwe)

- Nazwa testu = “kto + co + dowód”
- Przykład:
  - `@smoke Zakup: user finalizuje koszyk i widzi potwierdzenie`
- Pliki:
  - `checkout.spec.ts`
  - `auth.spec.ts`
- Zasada: 1 plik = 1 obszar funkcjonalny

Checklist: PR review testów Playwright

- Czy test ma 1 jasny cel?
- Czy kroki są w AAA i krótkie?
- Czy locatory są stabilne (role/label/testid)?
- Czy expected results są jednoznaczne?
- Czy są 2 asercje dowodowe?
- Czy brak sleep i hacków timeout?
- Czy test jest niezależny (dane, konto, kolejność)?
- Czy artefakty (trace/screenshot) pomogą w diagnozie?

BAD vs GOOD #1: locatory i asercje (login)

- Ten sam scenariusz
- Różnica: stabilność + dowód jakości

```
// ❌ BAD: kruche locatory + słaba asercja
import { test, expect } from '@playwright/test';

test('login', async ({ page }) => {
  await page.goto('/login');
  await page.locator('input[name="email"]').fill('a@b.com');
  await page.locator('input[name="password"]').fill('pass');
  await page.locator('.btn.primary').click();
  await expect(page.locator('div')).toBeVisible(); // nic nie udowadnia
});
```

BAD vs GOOD #1: locatory i asercje (login)

- Ten sam scenariusz
- Różnica: stabilność + dowód jakości

```
// ✅ GOOD: role/label + 2 asercje dowodowe
import { test, expect } from '@playwright/test';

test('@smoke Logowanie: user widzi dashboard', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('a@b.com');
  await page.getByLabel('Hasło').fill('pass');
  await page.getByRole('button', { name: 'Zaloguj' }).click();

  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

BAD vs GOOD #2: flaky waiting (zapis profilu)

- Różnica: “magiczny timeout” vs czekanie na stan

```
// ❌ BAD: waitForTimeout maskuje problem
import { test, expect } from '@playwright/test';

test('save profile', async ({ page }) => {
  await page.goto('/profile');
  await page.getByRole('button', { name: 'Zapisz' }).click();
  await page.waitForTimeout(1500); // czasem za mało / czasem niepotrzebne
  await expect(page.getByText('Zapisano')).toBeVisible();
});
```

BAD vs GOOD #2: flaky waiting (zapis profilu)

- Różnica: “magiczny timeout” vs czekanie na stan

```
// ✅ GOOD: czekamy na stan UI lub odpowiedź
import { test, expect } from '@playwright/test';

test('@regression Profil: zapis pokazuje status', async ({ page }) => {
  await page.goto('/profile');

  const save = page.waitForResponse(r =>
    r.url().includes('/api/profile') && r.request().method() === 'PUT' && r.ok()
  );

  await page.getByRole('button', { name: 'Zapisz' }).click();
  await save;

  await expect(page.getByRole('status')).toContainText('Zapisano');
  await expect(page.getByRole('button', { name: 'Zapisz' })).toBeEnabled();
});
```

Ćwiczenie: “Napraw BAD do GOOD”

- Uczestnicy dostają 2 pliki BAD
- Zadanie:
  1. refaktor locatorów
  2. poprawa asercji
  3. usunięcie sleep
  4. dodanie tagów @smoke/@regression
- Narzędzia:
  - Copilot do refaktoru
  - (opcjonalnie) MCP do review

Kryteria zaliczenia

- brak `waitForTimeout`
- locatory wg policy
- 2 asercje dowodowe
- test niezależny
- commit message: “stabilize + strengthen asserts”

Podsumowanie: “proces > narzędzie”

- Playwright daje narzędzia jakości (trace, expect, locatory)
- Copilot/MCP przyspieszają:
  - generowanie
  - review
  - refaktor
- Powtarzalność daje:
  - policy + checklist + ćwiczenia + triage flaky

AI w procesie deweloperskim

hype a rzeczywistość

Vibe coding

"Kodowanie intuicyjne"

W praktyce vibe coding przyjmuje dwie formy:

Czysty vibe coding

– pełne zaufanie do wyników AI.

Sprawdza się przy szybkich eksperymentach lub „weekendowych projektach do wyrzucenia”, gdzie liczy się tempo, a nie perfekcja.

Odpowiedzialne programowanie z asystą AI

– AI działa jak inteligentny współprogramista.

Ty prowadzisz, weryfikujesz, testujesz i rozumiesz wygenerowany kod. To Ty zachowujesz kontrolę i pełną odpowiedzialność za efekt końcowy.

Vibe coding

"Kodowanie intuicyjne"

Najpopularniejsze narzędzia do „ai - codingu”:

- **GitHub Copilot** – asystent AI w IDE, podpowiada kontekstowo kod. Obsługa kontekstu, MCP, itd.
- **Cursor** – (fork VS Code) z generowaniem i debugowaniem kodu.
- **WindSurf** – edytor AI do automatyzacji DevOps i pracy w chmurze.
- **Replit** – chmurowe IDE z funkcją AI „Get Unstuck”.
- **Cody (Sourcegraph)** – asystent AI z dostępem do repozytorium.
- **Bolt (StackBlitz)** – generuje aplikacje full-stack.
- **v0 (Vercel)** – generuje UI (React + Tailwind)
- **ChatGPT** – uniwersalny asystent AI do generowania i debugowania kodu.

Vibe Coding

– skrócona checklista dobrych praktyk

- Najpierw Wizja i projekt
  - Wiedz, co i po co budujesz.
  - Myśl z perspektywy użytkownika i produktu.
  - Najpierw UI/UX i spójny system komponentów.

- Środowisko i stack
  - Używaj popularnych, dobrze wspieranych technologii (AI zna je najlepiej)
  - Folder /instructions z przykładami i zasadami
  - Czyste repozytorium GIT i często commity .

- Praca z AI
  - Pisz konkretne, jednoznaczne prompty.
  - Dziel duże funkcje na małe kroki.
  - Restartuj chat, gdy AI zaczyna się gubić
  - Podawaj tylko istotne pliki i komponenty, odwołuj się do przykładów.
  - Audytuj kod, Waliduj dane, ukrywaj sekrety, weryfikuj uprawnienia i błędy

- Mindset
  - Jasność i struktura ponad szybkość.
  - Krótkie pętle, szybka weryfikacja.
  - Buduj, testuj, poprawiaj — vibe z dyscypliną.

Kilka praktycznych wskazówek

- Najpierw opowiedz mi swój plan — nie pisz kodu.
- Podaj kilka opcji, zaczynając od najprostszej — bez kodowania.
- Pomyśl tyle, ile trzeba, i zadawaj pytania, jeśli potrzebujesz więcej informacji.
- Wyszukaj powtórzenia lub zbędny kod i wypisz je.
- Skup się na zrozumieniu, jak działa kod, a nie na nauce jego pisania.

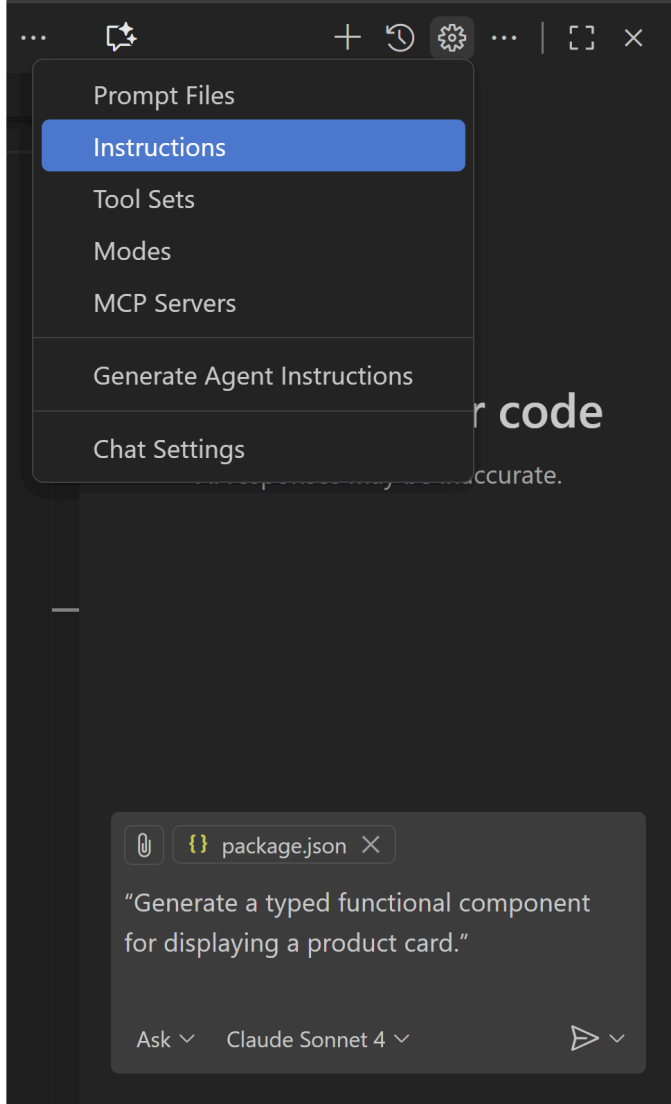
Copilot Chat

Okno Copilot Chat (panel boczny lub inline)

- /explain — wyjaśnij zaznaczony kod
- /fix — popraw błędy lub linter issues
- /tests — wygeneruj testy jednostkowe
- /docs — stwórz dokumentację lub komentarz
- /commit — zaproponuj opis do commita

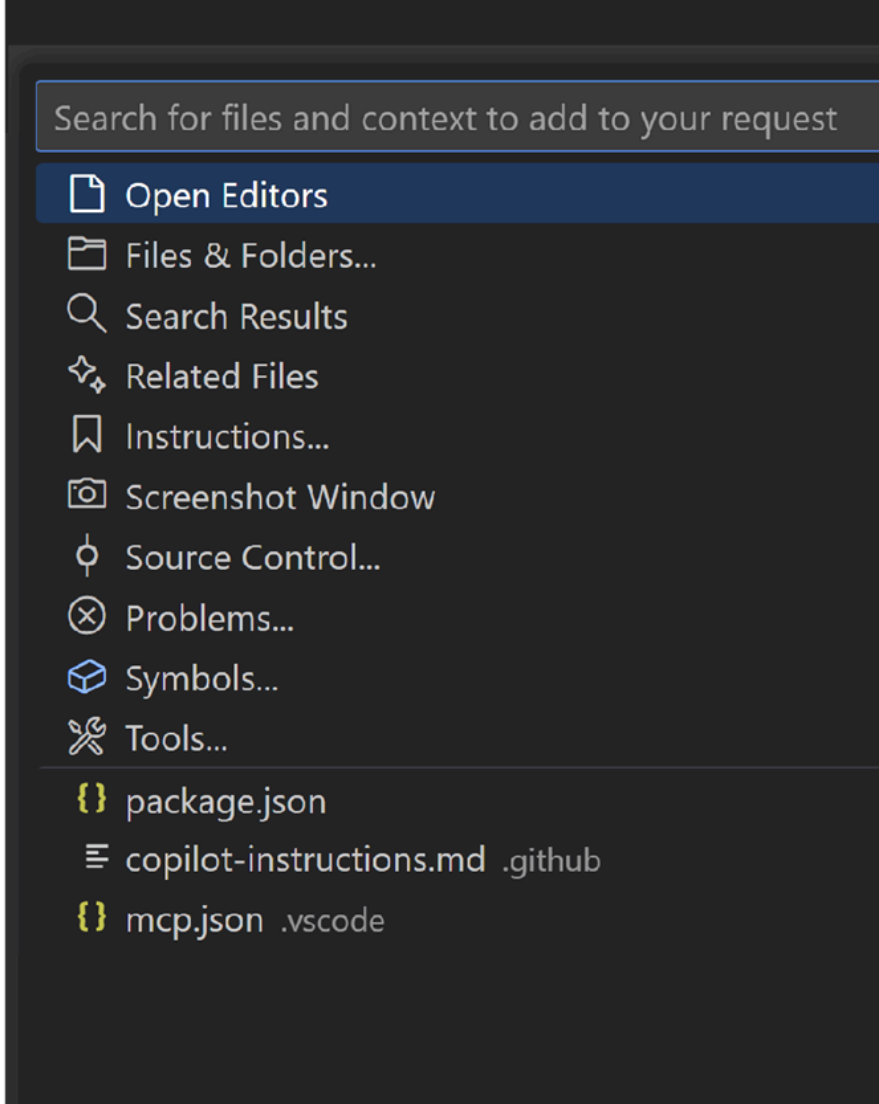
Inline Chat (Ctrl+I) - Chat kontekstowy otwierany w edytorze obok zaznaczonego fragmentu kodu.

- wyjaśnienie działania
- refaktoryzację
- konwersję do TypeScript
- dodanie testów lub komentarzy



Konteksty Copilot Chat

- Open Editors – aktualnie otwarte pliki.
- Files & Folders – wybrane pliki lub katalogi.
- Search Results – wyniki wyszukiwania.
- Related Files – powiązane testy, style, typy.
- Instructions – plik zasad i stylu (copilot-instructions.md).
- Screenshot Window – zrzut ekranu edytora.
- Source Control – zmiany z Git (diff, commity).
- Problems – błędy i ostrzeżenia z panelu Problems.
- Symbols – funkcje, klasy i komponenty z kodu.
- Tools – integracje z narzędziami (npm, eslint, git).





Process AI-driven

draft → refine → verify → integrate

Wprowadzenie

do programowania z AI

Czym jest proces AI-driven

- Systematyczny sposób pracy z Copilotem
- Oparty na cyklu iteracyjnym,
  nie pojedynczym promptcie
- Łączy kreatywność AI z kontrolą programisty

Dlaczego potrzebny jest proces

- Pojedynczy prompt = losowy wynik
- Proces = przewidywalne rezultaty i wyższa jakość
- Każdy etap ma inne pytania i metody

Rola Copilota

- **Partner**, nie generator
- Pomaga tworzyć, tłumaczyć, testować i dokumentować
- Wymaga kontekstu i kierunku, nie zgadywania

Draft

Uzyskaj pierwszą wersję kodu lub struktury, szybko i bez oceniania

Prompty:

- “Create base React component with placeholder data”
- “Generate folder structure for a X feature using A + B
- “Bootstrap simple layout with header and sidebar using Z”

Kroki:

- Poproś Copilota o wygenerowanie pojedynczego komponentu
- Sprawdź, co stworzył, bez poprawiania
- Zapisz wersję jako `draft`

Refine

Uporządkuj, doprecyzuj i popraw jakość kodu

Prompty:

- “Refactor into smaller components and add typings”
- “Improve readability and follow our style rules”
- “Add absract types and interfaces to this component”
- “Extract data fetching logic into custom service"

Kroki:

Ulepsz Komponent dodając typy i logikę stanu

Verify

Sprawdź poprawność, stabilność i jakość

Prompty:

- “Generate test for this component”
- “Check this file for performance or security issues”
- “List potential runtime errors and fix them”

Kroki:

- Wygeneruj test dla komponentu
- Uruchom npm test
- Popraw błędy wskazane przez Copilot Chat

Integrate

Połącz dopracowane elementy w spójny system

Prompty:

- “Summarize this diff as a commit message”
- “Document public API for this module”
- “Integrate components and services into complete view”
- “Explain how state flows through this component”

Kroki:

- Zintegruj komponent i serwis
- Wygeneruj opis commita przez Copilota
- Sprawdź integrację aplikacji

Copilot w Pull Requestach

Code-review na "naszych zasadach"

Zasada ogólna

Copilot “czyta” tylko to, co ma w kontekście:

- pliki repozytorium
- diff PR
- opis PR
- komentarze w PR

➡️ Twoje zasady muszą:

- istnieć w repo
- pojawiać się w kontekście PR.

Zasady w repo – źródło prawdy

Umieść wytyczne jako krótkie pliki:

- `docs/architecture`
  - `architecture-guidelines.md`
  - `bounded-contexts.md`
- `docs/coding-style`
  - `typescript-style-guide.md`
- `docs/testing`
  - `testing-strategy.md`
- `docs/security`
  - `secure-coding-guide.md`

Dlaczego działa?
Copilot widzi pliki repo i potrafi je wykorzystać przy generowaniu zmian.

Włącz zasady do szablonu PR

`PULL_REQUEST_TEMPLATE.md` jako mini-prompt:

```
## Cel zmian

<!— Krótko opisz problem i rozwiązanie. —>

## Kontekst architektoniczny

- [ ] Zgodne z naszym stylem architektonicznym (`docs/architecture/architecture-guidelines.md`)
- [ ] Zgodne z zasadami warstw (`docs/architecture/layers.md`)
- [ ] Zgodne z zasadami DDD (`docs/architecture/ddd-rules.md`)

## Checklist

- [ ] Zastosowano konwencje nazewnicze z `docs/coding-style/typescript-style-guide.md`
- [ ] Dodano testy zgodnie z `docs/testing/testing-strategy.md`
- [ ] Zwrócono uwagę na bezpieczeństwo (`docs/security/secure-coding-guide.md`)
```

Komentarze w PR jako

“mini system prompts”

Używaj komentarzy typu:

- „Zrefaktoryzuj zgodnie z `ddd-rules.md`”
- „Proponuj zmiany zgodne z `typescript-style-guide.md`”
- „Zastosuj nasze zasady warstwowania (patrz `layers.md`)”

Copilot doładuje te pliki do kontekstu.

Połącz Copilota z twardymi regułami

Copilot to asystent, nie policjant.

Egzekwuj zasady narzędziowo:

- ESLint / Prettier
- ArchUnit, dependency rules
- testy architektoniczne
- skanery bezpieczeństwa

Copilot pomaga „naprawić”,
ale to CI wymusza zgodność.

Narzędzia do statycznej analizy kodu są:

- przewidywalne
- konfigurowalne
- wytłumaczalne
- ścisłe

- ...
- AI nie jest.

Copilot Enterprise + baza wiedzy

Jeśli organizacja używa Copilot Enterprise:

- dodaj tam ADR-y, guideline’y, playbooki
- Copilot w PR będzie mógł się do nich odwołać
- zwiększa trafność refaktoryzacji architektonicznych

Standaryzowane prompty dla reviewerów

Przykłady komentarzy:

- „Przeanalizuj PR pod kątem warstw i DDD (patrz `architecture/layers.md`).”
- „Sprawdź zgodność ze style guide + zaproponuj brakujące testy.”
- „Zidentyfikuj naruszenia zasad bezpieczeństwa.”

Stosowane konsekwentnie → Copilot zaczyna trafiać częściej.

CODEOWNERS

AI review gate

Wzmocnij proces PR:

- `CODEOWNERS` dla kluczowych katalogów
- CI z linterami, arch-tests, security scan
- Copilot wspiera autora i architekta w przejściu review

CODEOWNERS

```
# This is a comment.

*       @username1

/docs   @username2

*.js    @dev-team
```

Gwiazdka `*` reprezentuje wszystkie pliki w repozytorium, a `@username1` jest przypisany jako właściciel kodu.

`/docs` określa, że `@username2` będzie właścicielem kodu wszystkich plików w katalogu docs.

`*.js` wskazuje, że wszelkie pliki JavaScript w repozytorium należą do `@dev-team`.

Plan wdrożenia

w organizacji:

- Stwórz krótkie zasady w `docs/`
- Dodaj szablon PR z checklistami
- Ustaw linters + arch-tests
- Przygotuj zestaw komentarzy-promptów
- Przeszkol zespół (1h)
- (Opcjonalnie) Połącz z Copilot Enterprise

Efekt

Copilot zaczyna proponować zmiany:

- zgodne z architekturą
- spójne ze stylem organizacji
- zgodne z zasadami DDD, testów i bezpieczeństwa

➡️ PR stają się szybsze, czytelniejsze i bardziej spójne.